

DWWM 2025-2026 — Titre professionnel

Développeur web et web mobile

DOSSIER PROJET

PopJav

Plateforme d'apprentissage du langage Java

Enzo Gavini

Eureka / La Plateforme — Campus des innovations de Martigues

Soutenance : 5 août 2026



Sommaire

01 — Introduction.....	3
02 — Compétences mises en œuvre.....	4
03 — Résumé du projet.....	8
04 — Gestion de projet.....	9
05 — Conception fonctionnelle.....	10
06 — Conception visuelle.....	14
07 — Conception technique.....	21
08 — Architecture.....	26
09 — Développement : extraits de code commentés.....	29
10 — Sécurité de l'application.....	32
11 — Conteneurisation et déploiement.....	35
12 — Tests.....	36
13 — Recette et validation.....	38
14 — Conformité RGPD et accessibilité (RGAA).....	42
15 — Veille technologique et sécurité.....	43
16 — Difficultés rencontrées et évolutions futures.....	44
17 — Conclusion.....	45
18 — Annexes.....	46

01 — Introduction

Ce dossier présente PopJav, une plateforme d'apprentissage du langage Java que j'ai conçue et développée seul, de la première maquette au déploiement conteneurisé, dans le cadre de la formation Développeur Web et Web Mobile, suivie en présentiel au Campus des innovations de Martigues, de juin 2025 à juillet 2026.

Cette formation marque le début de ma reconversion professionnelle dans le développement informatique. Elle est la première étape d'un parcours que je compte poursuivre avec le titre de Concepteur Développeur d'Applications (CDA), et pourquoi pas jusqu'au master.

PopJav est né d'un constat simple : apprendre un langage de programmation comme Java demande de la régularité, et la théorie seule décourage vite. J'ai voulu construire une plateforme qui rende cette montée en compétences progressive et ludique, en s'inspirant des mécaniques de jeu — des vies, des scores, des seuils à franchir — appliquées à un vrai parcours pédagogique.

Je remercie les formateurs de La Plateforme pour leur accompagnement et leur exigence tout au long de cette formation, ainsi que mes camarades de promotion pour l'entraide et les moments partagés.

02 — Compétences mises en œuvre

Les compétences du référentiel couvertes par le projet PopJav sont :

Pour l'activité type 1 : développer la partie front-end d'une application web ou web mobile sécurisée

Installer et configurer son environnement de travail en fonction du projet web ou web mobile

Mon environnement de développement repose sur IntelliJ IDEA, configuré avec le JDK Amazon Corretto 17 (version LTS). PopJav étant découpé en sept microservices, chacun possède son propre pom.xml Maven qui centralise quatre choses fondamentales : la gestion des dépendances, un cycle de vie de build standardisé, un build reproductible grâce au Maven Wrapper, et l'intégration des outils de test (JUnit partout, complété par JaCoCo sur les modules porteurs de logique métier) et de déploiement (Jib).

Le versionnement est assuré par Git, avec un dépôt distant sur GitHub et la convention Conventional Commits (feat:, fix:, docs:, test:...) pour garder un historique lisible ; les fonctionnalités sont développées sur des branches dédiées puis fusionnées dans main.

Enfin, l'infrastructure locale est conteneurisée avec Docker : PostgreSQL, MongoDB et Consul tournent dans des conteneurs créés à partir des images officielles, configurés par des variables d'environnement centralisées dans un fichier .env — les services applicatifs ont été ajoutés à cette orchestration au fur et à mesure du développement.

Maquetter des interfaces utilisateur web ou web mobile

À partir des besoins du projet, j'ai d'abord réalisé les wireframes des écrans clés — catalogue des chapitres, page de leçon, quiz interactif, profil et pages d'administration — puis je les ai déclinés en maquettes détaillées sur Figma.

J'ai appliqué l'approche mobile-first : chaque écran a été pensé d'abord pour mobile, puis décliné en version desktop, afin de garantir une interface adaptée à tous les types de supports. Ces maquettes ont ensuite servi de référence pendant toute la phase d'intégration en Thymeleaf et Tailwind CSS.

Réaliser des interfaces utilisateur statiques web ou web mobile

Pour ce projet, j'ai développé l'ensemble des pages en HTML5 avec CSS3 et le framework Tailwind CSS. J'ai appliqué une palette de couleurs et des typographies cohérentes (Baloo 2 pour les titres, Poppins pour l'interface et les textes), et mes feuilles de style contiennent des variables CSS personnalisées pour garantir la cohérence visuelle sur l'ensemble du site.

Les éléments communs à toutes les pages — en-tête, barre de navigation, pied de page — sont factorisés en fragments Thymeleaf réutilisables : chaque page les inclut au lieu de les dupliquer,

ce qui centralise la maintenance. Les feuilles de style sont versionnées (style.css?v=6) pour forcer le rafraîchissement du cache des navigateurs à chaque évolution.

En suivant mes maquettes, j'ai décliné chaque écran en responsive pour qu'il s'adapte à tous les formats — mobile, tablette et desktop. J'ai utilisé Flexbox pour la structure générale (navigation, cartes, formulaires) et CSS Grid pour la grille de réponses du quiz : une colonne sur mobile, deux colonnes sur grand écran via une simple media query — une illustration directe de l'approche mobile-first. Pendant le développement, je validais régulièrement l'affichage sur plusieurs navigateurs.

Développer la partie dynamique des interfaces utilisateur web ou web mobile

La partie dynamique de PopJav repose d'abord sur Thymeleaf : les pages sont générées côté serveur à partir des données réelles — listes de chapitres et de leçons parcourues avec `th:each`, contenu des leçons, statistiques du profil calculées à partir des résultats de l'utilisateur. Les formulaires sont liés aux objets Java avec `th:object` et `th:field` : Thymeleaf gère la liaison des champs dans les deux sens, de l'affichage à la soumission.

L'affichage s'adapte ensuite à l'utilisateur connecté grâce à la session : les actions d'administration (créer, modifier, supprimer) ne sont affichées qu'aux comptes ADMIN via des conditions `th:if` sur le rôle, et la barre de navigation change selon que le visiteur est connecté ou non. L'écran le plus dynamique est le quiz : le formulaire de jeu est généré à partir des questions et réponses du quiz choisi, et la page de résultat affiche le score, les vies restantes et la correction détaillée question par question.

Enfin, quatre scripts JavaScript complètent l'expérience côté navigateur. Le plus complet pilote le quiz : les questions s'affichent une par une avec des boutons Précédent/Suivant et un compteur de progression, et un minuteur de 30 secondes par question soumet automatiquement le quiz quand le temps est écoulé — les questions sans réponse comptent alors comme fausses. Le menu burger rend la navigation utilisable sur mobile, en tenant l'attribut `aria-expanded` à jour pour les lecteurs d'écran. La validation de formulaires vérifie les champs avant soumission (champs vides, format d'email, politique de mot de passe à l'inscription) et affiche les erreurs dans une zone accessible (`role="alert"`, `aria-invalid`) ; cette validation cliente améliore le confort mais ne remplace pas la validation serveur, qui applique exactement les mêmes règles et reste la seule garantie de sécurité. Un dernier script demande confirmation avant toute suppression, pour éviter les pertes de données accidentelles.

Pour l'activité type 2 : développer la partie back-end d'une application web ou web mobile sécurisée

Mettre en place une base de données relationnelle

J'ai conçu la base de données relationnelle de PopJav en suivant la démarche classique de modélisation : le modèle conceptuel (MCD) à partir des besoins fonctionnels, puis le modèle logique (MLD) et enfin le modèle physique (MPD) sous PostgreSQL. Le schéma s'articule autour

des utilisateurs, de la hiérarchie de contenu pédagogique — un chapitre contient des leçons ordonnées, une leçon peut porter un quiz, composé de questions et de réponses — et des résultats de quiz qui tracent la progression de chaque utilisateur.

Les règles d'intégrité sont portées par la base elle-même : clés étrangères entre les tables de contenu, et contraintes d'unicité sur l'email et le nom d'utilisateur — même si un contrôle applicatif était contourné, la base refuserait un doublon.

Cette base est déployée dans un conteneur Docker à partir de l'image officielle PostgreSQL 16, avec un volume pour la persistance des données, un healthcheck (pg_isready) qui conditionne le démarrage des services qui en dépendent, et une configuration (identifiants, port) externalisée dans un fichier .env. Seul le microservice persistance-service communique avec elle.

Développer des composants d'accès aux données SQL et NoSQL

Côté SQL, l'accès aux données est réalisé avec Spring Data JPA (Hibernate) dans persistance-service : chaque entité (User, Chapter, Lesson, Quiz, Question, Answer, Result) possède son repository, une interface dont Spring génère l'implémentation. Les méthodes dérivées comme findByEmail ou existsByUsername sont traduites automatiquement en requêtes SQL préparées : les paramètres sont liés séparément de la requête, ce qui protège l'application des injections SQL sans écrire une seule ligne de SQL à la main.

Côté NoSQL, les commentaires des utilisateurs sont gérés par comment-service avec Spring Data MongoDB : la classe Comment, annotée @Document, est persistée sous forme de document, et son repository hérite de MongoRepository, avec des méthodes dérivées comme findById ou findByLessonId. J'ai choisi MongoDB pour ce cas d'usage car un commentaire est un document autonome, au schéma souple, sans jointures — le modèle documentaire y est plus adapté que le relationnel.

Les deux mondes sont reliés par des références logiques : le document Comment stocke les identifiants de l'utilisateur et de la leçon sous forme numérique, sans clé étrangère physique — aucune FK n'est possible entre deux bases hétérogènes. C'est la logique applicative et la chaîne d'identité du gateway qui garantissent la cohérence.

Comme PostgreSQL, MongoDB est déployé dans un conteneur Docker (image officielle, volume de persistance, healthcheck, configuration externalisée dans le .env), et seul comment-service communique avec lui.

Développer des composants métier côté serveur

J'ai développé les composants métier de PopJav en Java avec Spring Boot, en respectant une architecture en couches : les controllers reçoivent les requêtes HTTP et les délèguent aux services, qui portent la logique métier et s'appuient sur les repositories pour l'accès aux données. Cette séparation des responsabilités, conforme aux principes de la programmation orientée objet, rend chaque couche testable et remplaçable indépendamment.

La logique métier la plus riche est le moteur de quiz. À la soumission d'une partie, le service récupère sa propre copie du quiz — on ne fait jamais confiance à un score calculé par le client — puis évalue les réponses une à une : chaque erreur coûte une vie, la partie s'arrête immédiatement à zéro vie, et le score final est un pourcentage comparé au seuil de réussite du quiz, avec une garde contre la division par zéro pour un quiz sans questions. Symétriquement, avant tout envoi au client, les bonnes réponses sont masquées : un joueur qui inspecterait la réponse réseau n'y trouverait aucune solution.

Le service d'authentification porte une autre logique sensible : validation de la politique de mot de passe côté serveur, vérification de l'unicité de l'email et du nom d'utilisateur, hachage BCrypt avant tout envoi vers la persistance, et message d'erreur générique à la connexion pour ne pas révéler si un compte existe.

Les cas d'erreur sont gérés par des exceptions métier typées (`EmailAlreadyExistsException`, `InvalidCredentialsException`, `ResourceNotFoundException`...), converties par des handlers globaux en codes HTTP appropriés : 409 pour un conflit, 401 pour des identifiants invalides, 404 pour une ressource introuvable. Enfin, les services sont testés unitairement avec JUnit 5 et Mockito — les dépendances externes sont simulées pour isoler la logique testée — avec une couverture de 88 à 89 % mesurée par JaCoCo, et le code est documenté en anglais (javadoc de classes et commentaires sur la logique métier et de sécurité).

Documenter le déploiement d'une application dynamique web ou web mobile

PopJav est composé de sept microservices qui se découvrent dynamiquement via Consul et communiquent entre eux en REST. Pour rendre ce déploiement reproductible, chaque service est empaqueté en image Docker avec Jib, un plugin Maven qui construit des images optimisées sans Dockerfile ; les images sont publiées sur Docker Hub.

Un fichier `docker-compose` unique orchestre la plateforme complète — dix conteneurs : les sept services applicatifs, PostgreSQL, MongoDB et Consul. L'ordre de démarrage est maîtrisé par des `healthchecks` : les services applicatifs n'attendent pas simplement que les bases soient lancées, mais qu'elles soient réellement prêtes à répondre. Toute la configuration sensible (ports, identifiants des bases, secret JWT) est externalisée dans un fichier `.env`, jamais dans les images ni dans le code.

J'ai documenté l'ensemble dans le README du dépôt GitHub, rédigé en anglais : prérequis techniques, description de l'architecture avec ses diagrammes, variables d'environnement à renseigner (un `.env.example` sert de modèle), et procédure de lancement étape par étape — l'application complète démarre en deux commandes (`cp .env.example .env` puis `docker compose up -d`).

03 — Résumé du projet

3.1 Concept

PopJav est une plateforme web d'apprentissage du langage Java qui rend la montée en compétences progressive et ludique. Le contenu est organisé en chapitres, composés de leçons, chacune pouvant être validée par un quiz « à vies » inspiré des jeux vidéo : chaque mauvaise réponse coûte une vie, et la partie s'arrête quand il n'en reste plus. Un score en pourcentage et un seuil de réussite par quiz permettent de mesurer la progression, consultable depuis le profil.

3.2 Fonctionnalités par rôle

Un visiteur non connecté peut :

- Découvrir le catalogue public des chapitres (métadonnées uniquement, le contenu des leçons reste protégé).
- Créer un compte via un formulaire sécurisé, puis se connecter.

Un utilisateur connecté (USER) peut :

- Lire les leçons, chapitre par chapitre.
- Jouer les quiz : vies limitées, score en pourcentage, correction détaillée en fin de partie.
- Commenter les leçons — l'identité de l'auteur est imposée côté serveur, jamais reprise du formulaire.
- Consulter son profil : statistiques de progression, quiz joués et réussis.

Un administrateur (ADMIN) peut :

- Gérer tout le contenu pédagogique : création, modification et suppression des chapitres, leçons, quiz, questions et réponses.
- Modérer les commentaires : la suppression est réservée au rôle ADMIN.
- Lister et administrer les comptes utilisateurs, via l'API sécurisée du gateway (routes réservées au rôle ADMIN).

04 — Gestion de projet

4.1 Méthodologie et planning

J'ai travaillé de manière itérative, fonctionnalité par fonctionnalité : chaque itération part d'un besoin, passe par le développement et les tests, puis est fusionnée dans la branche principale via des branches thématiques (feat/..., docs/..., test/...). L'historique Git constitue le journal de bord réel du projet et permet de reconstituer les grandes phases :

- Phase 1 — Socle : mise en place des 7 microservices, du service discovery Consul et des CRUD de base.
- Phase 2 — Métier : moteur de quiz (vies, score, seuil), parcours utilisateur complet.
- Phase 3 — Sécurité : JWT, RBAC au gateway, propagation d'identité, fermeture des failles IDOR, hachage BCrypt.
- Phase 4 — Qualité : exceptions typées, tests unitaires JUnit/Mockito, couverture JaCoCo de 88-89 %.
- Phase 5 — Interface : responsive, accessibilité, catalogue public, pages légales.
- Phase 6 — Industrialisation : conteneurisation Jib, docker-compose complet, README et diagrammes.
- Phase 7 — Documentation : commentaires de code en anglais sur l'ensemble du projet.

4.2 Environnement de travail

Outils utilisés : IntelliJ IDEA (développement), Git/GitHub (versionnement, branches et merges), Maven (build), Docker Desktop (infrastructure locale), Figma (maquettes), Postman (tests d'API). Environnement humain : projet individuel, accompagné par les formateurs de La Plateforme, avec des échanges réguliers au sein de la promotion.

05 — Conception fonctionnelle

5.1 Cahier des charges et priorisation MoSCoW

Les fonctionnalités ont été priorisées selon la méthode MoSCoW :

Priorité	Fonctionnalités
Must have	Inscription/connexion sécurisées ; catalogue de chapitres ; lecture des leçons ; quiz à vies avec score et seuil de réussite ; rôles USER/ADMIN ; CRUD du contenu (admin)
Should have	Commentaires sur les leçons ; profil avec statistiques de progression ; catalogue public pour les visiteurs ; pages légales
Could have	Badges et gamification avancée ; éditeur de contenu riche ; tableau de bord admin dédié
Won't have (v1)	Paieement / abonnement ; application mobile native ; mode hors-ligne

5.2 User stories

ID	User story	Priorité
US-01	En tant que visiteur, je veux consulter le catalogue des chapitres afin de découvrir le contenu avant de m'inscrire.	Must
US-02	En tant que visiteur, je veux créer un compte afin d'accéder aux leçons.	Must
US-03	En tant qu'utilisateur, je veux me connecter afin de retrouver ma progression.	Must
US-04	En tant qu'utilisateur, je veux lire une leçon afin d'apprendre une notion de Java.	Must
US-05	En tant qu'utilisateur, je veux jouer un quiz avec des vies afin de tester mes connaissances de façon ludique.	Must
US-06	En tant qu'utilisateur, je veux voir la correction en fin de quiz afin de comprendre mes erreurs.	Must
US-07	En tant qu'utilisateur, je veux consulter mon profil afin de suivre mes statistiques (quiz joués, réussis).	Should
US-08	En tant qu'utilisateur, je veux commenter une leçon afin d'échanger avec les autres apprenants.	Should
US-09	En tant qu'administrateur, je veux créer/modifier/supprimer chapitres, leçons et quiz afin de faire vivre le contenu.	Must
US-10	En tant qu'administrateur, je veux lister les utilisateurs afin	Should

5.3 Parcours utilisateur et règles métier

Parcours type : arrivée sur le catalogue public → inscription → connexion → lecture d'une leçon → lancement du quiz associé → réponses question par question → écran de résultat (score, vies restantes, correction) → profil et statistiques.

Règles métier du quiz :

- Chaque quiz définit un nombre de vies et un seuil de réussite (pourcentage).
- Chaque mauvaise réponse coûte une vie ; à zéro vie, la partie s'arrête immédiatement et les questions restantes ne sont pas évaluées.
- Le score est un pourcentage de bonnes réponses sur le total de questions du quiz.
- Le quiz est réussi si le score atteint le seuil ; le résultat est enregistré dans l'historique de l'utilisateur.
- Les bonnes réponses ne sont jamais transmises au navigateur : le calcul se fait exclusivement côté serveur.
- Chaque question dispose de 30 secondes : à l'expiration du temps total, le quiz est soumis automatiquement et les questions restées sans réponse sont comptées fausses.



Quiz - Les variables

Vies : 3

1 / 3

1:28

Quel mot-clé déclare une constante en Java ?

☐ final

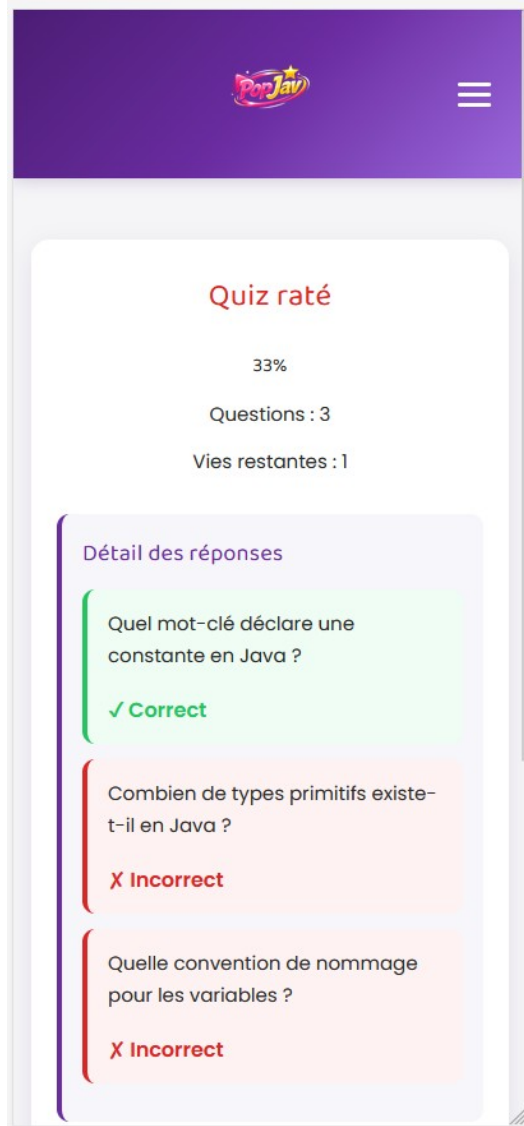
☐ const

☐ static

Suivant

PopJav

Le quiz en cours : vies, progression et minuteur



L'écran de fin de partie : score, vies restantes et correction détaillée

06 — Conception visuelle

6.1 Charte graphique



Le logo PopJav : l'énergie du rose et l'or de la récompense, avec l'étoile de la gamification

La palette de PopJav est définie en variables CSS personnalisées, ce qui garantit la cohérence sur tout le site et permettrait de changer de thème en un seul endroit :

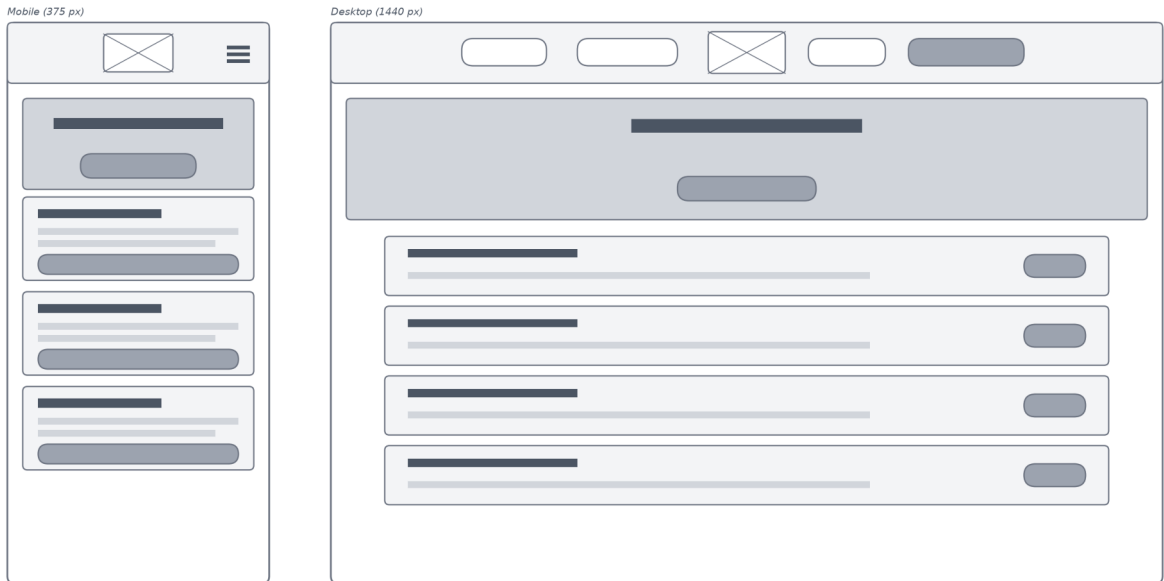
Couleur	Code	Usage et intention
Violet profond	#4B1D74 (dégradé vers #6A2CA0 et #9C6ADE)	Couleur identitaire : arrière-plans de mise en avant, en-têtes — un univers studieux mais moderne.
Rose vif	#F72585 (clair : #FF4FA3)	Couleur d'action : boutons principaux, éléments interactifs — l'énergie ludique de la plateforme.
Or	#FFC300 (clair : #FFD84D)	Récompense et progression (esprit gamification) ; c'est aussi la couleur de l'indicateur de focus clavier, très visible pour l'accessibilité.
Vert / Rouge	#22c55e / #dc2626	Feedback : bonne ou mauvaise réponse, succès ou erreur de formulaire.
Neutres	fond #F5F5F7, texte #2C2C2C	Lisibilité : un fond doux et un texte presque noir, pour un contraste confortable.

Typographies : Baloo 2 pour les titres (ronde et chaleureuse, elle porte le côté ludique) et Poppins pour l'interface et les contenus (géométrique et très lisible) — les deux sont déclarées en variables CSS (--font-titre, --font-corps). Les rayons arrondis (14 px, boutons « pilule ») et les ombres douces complètent cette identité accueillante.

6.2 Wireframes et maquettes

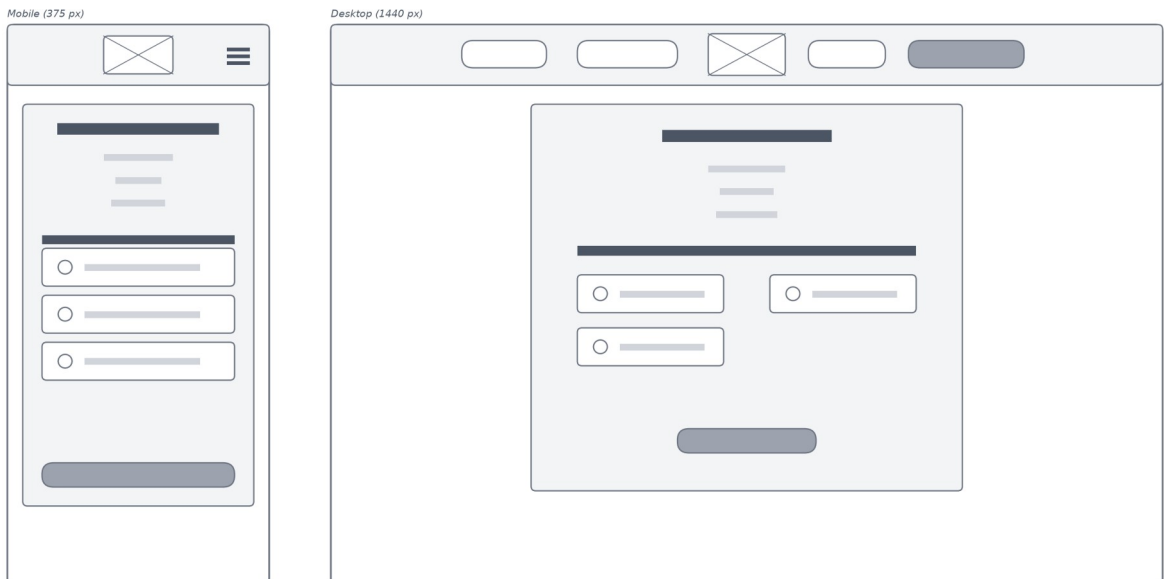
La conception a suivi deux étapes : des wireframes basse fidélité pour poser la structure de chaque écran (zones, hiérarchie, navigation), puis les maquettes détaillées qui appliquent la charte graphique. Deux wireframes représentatifs ci-dessous ; les autres figurent en annexe F.

Wireframe — Accueil (catalogue des chapitres)

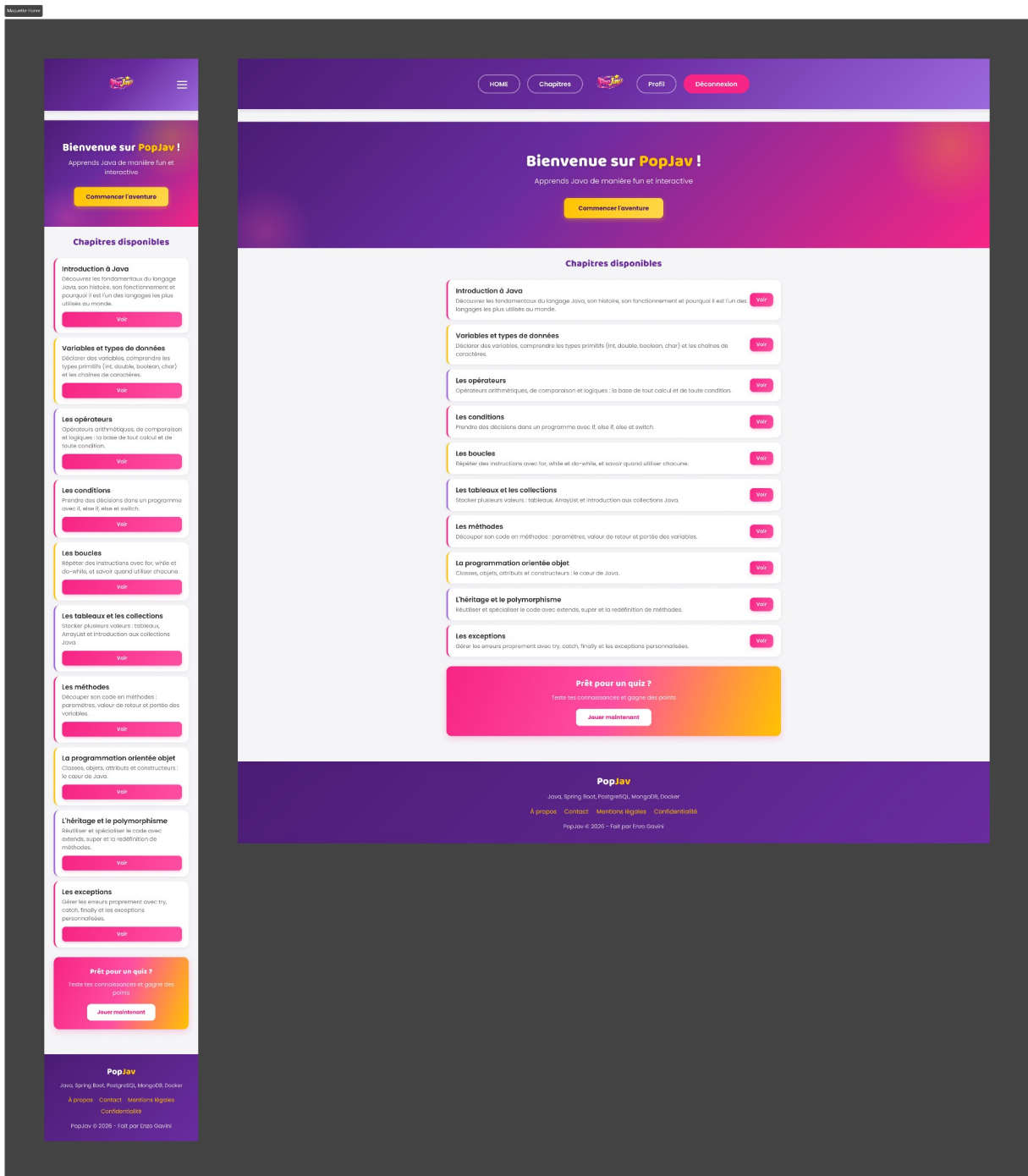


Wireframe de l'accueil — structure mobile et desktop

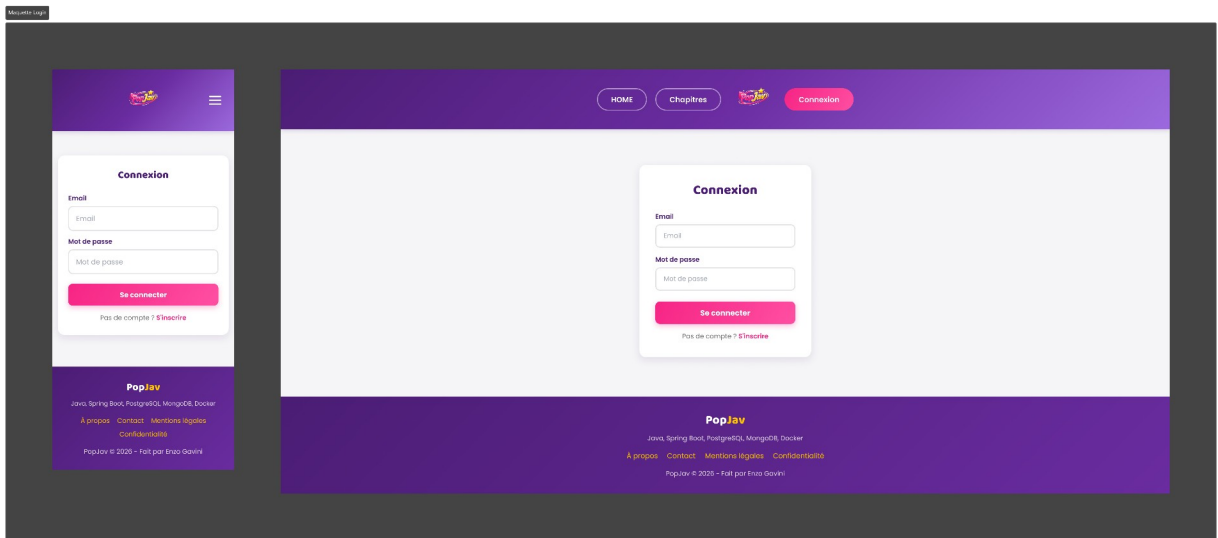
Wireframe — Quiz en cours (vies, progression, minuteur)



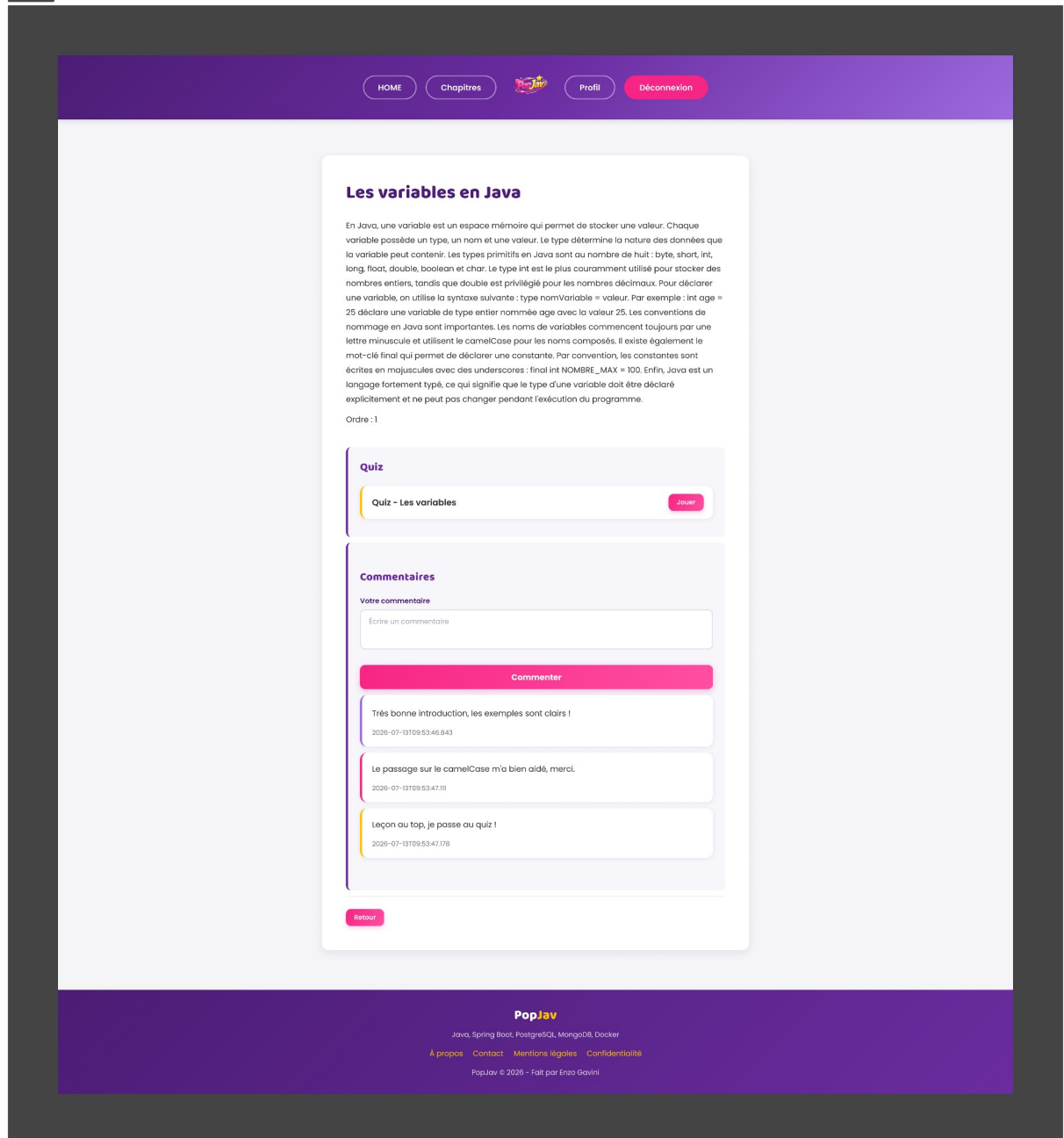
Wireframe du quiz — on y voit déjà la grille de réponses en deux colonnes sur desktop



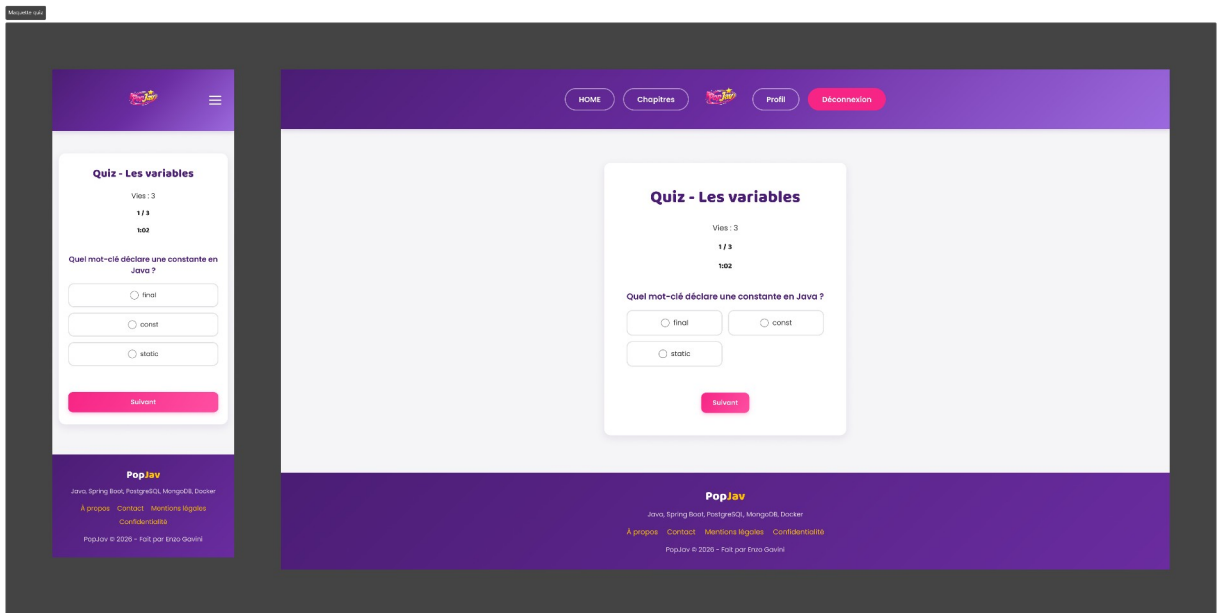
Maquette de l'accueil — versions mobile et desktop



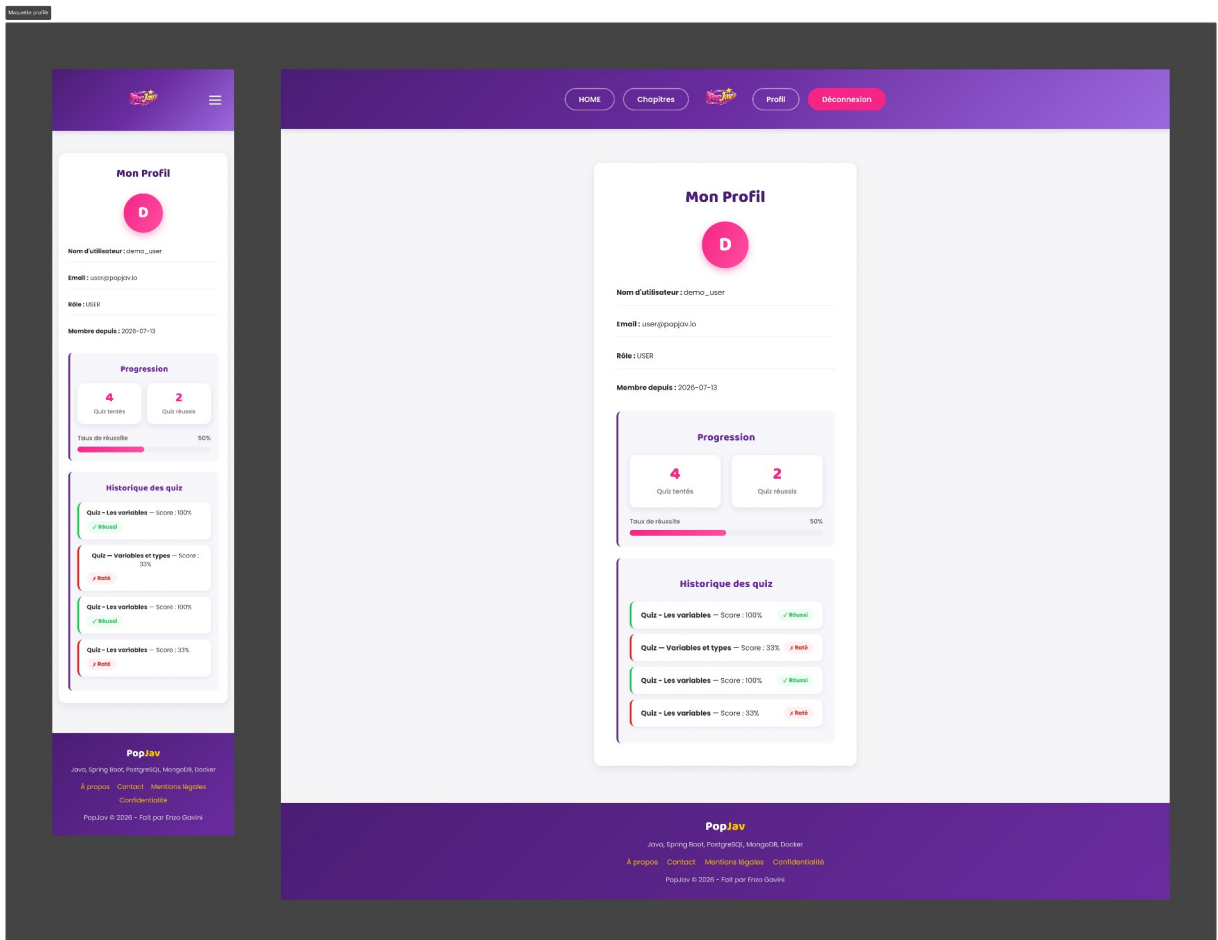
Maquette de la connexion — versions mobile et desktop



Maquette d'une page de leçon, avec le quiz associé et les commentaires



Maquette du quiz en cours — la grille de réponses passe d'une à deux colonnes selon l'écran



Les maquettes ont été réalisées sur Figma selon l'approche mobile-first (voir section 02) : chaque écran est pensé d'abord pour mobile puis décliné en desktop. Elles ont servi de référence pendant toute l'intégration.

6.3 Accessibilité — aperçu

L'accessibilité a été traitée dès l'intégration : étiquettes explicites sur tous les champs de formulaire, contrastes suffisants entre texte et fond, indicateur de focus clavier très visible (liseré or de 3 px), navigation complète au clavier, lien d'évitement vers le contenu principal (#main-content) sur toutes les pages, attributs alternatifs sur les images et attributs ARIA tenus à jour par les scripts (aria-expanded sur le menu burger, role="alert" et aria-invalid sur les erreurs de formulaire). Le détail de la conformité RGAA est présenté en section 14.

07 — Conception technique

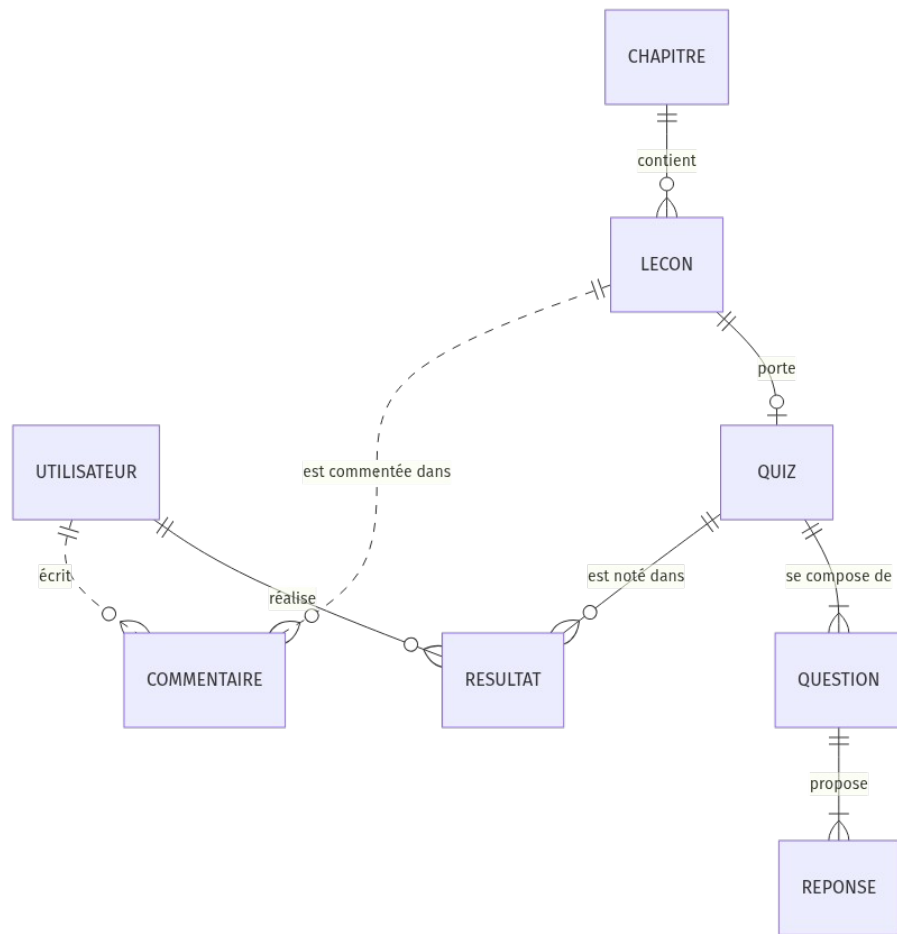
7.1 Environnement et technologies

Domaine	Technologies
Langage	Java 17 (Amazon Corretto)
Frameworks back-end	Spring Boot 3.2, Spring Cloud (Gateway, Consul Discovery, OpenFeign), Spring Security
Front-end	Thymeleaf (templates + fragments), Tailwind CSS, HTML5, JavaScript
Bases de données	PostgreSQL 16 (relationnel), MongoDB 7 (documents)
Accès aux données	Spring Data JPA / Hibernate, Spring Data MongoDB
Sécurité	JWT signés HS256 (JWT), BCrypt, RBAC au gateway
Tests	JUnit 5, Mockito, JaCoCo (88-89 % de couverture)
Build & dépendances	Maven (7 modules, Maven Wrapper)
Conteneurisation	Jib, Docker Hub, Docker Compose
Service discovery	HashiCorp Consul (+ healthchecks)
Outils	IntelliJ IDEA, Git/GitHub, Figma, Postman, Docker Desktop

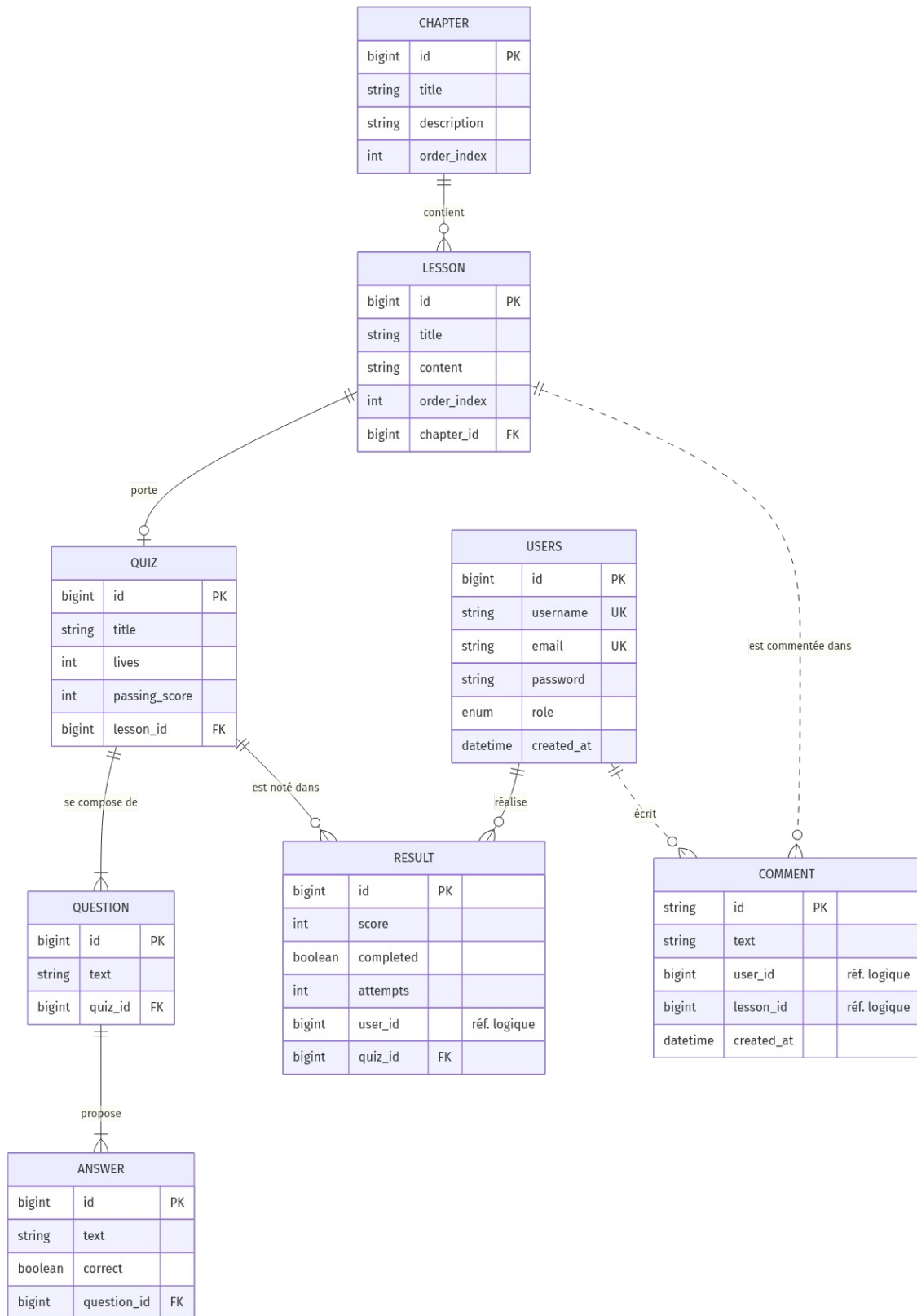
Le choix de Spring Boot s'explique par son écosystème complet : serveur web embarqué, configuration automatisée, injection de dépendances par annotations et intégration native de Spring Data, Spring Security et Spring Cloud. Maven structure le projet en sept modules et garantit un build reproductible sur n'importe quel poste grâce au Maven Wrapper.

7.2 Modélisation de la base de données

La conception est partie des besoins fonctionnels pour aboutir au modèle conceptuel (MCD), puis au modèle logique (MLD) et au modèle physique sous PostgreSQL. Le schéma s'articule autour de deux axes : le contenu pédagogique (un chapitre contient des leçons ordonnées ; une leçon peut porter un quiz ; un quiz contient des questions qui contiennent des réponses dont une seule est correcte) et le suivi (un résultat relie un utilisateur à un quiz avec son score, son statut de réussite et le nombre de tentatives).



Le MCD : les entités du domaine et leurs cardinalités, dans le vocabulaire métier — les liens pointillés sont les références logiques inter-contextes



Le MLD : les tables réelles avec leurs attributs, clés primaires et étrangères — les noms correspondent exactement au schéma PostgreSQL

Les contraintes d'intégrité sont portées par la base elle-même : clés étrangères entre les tables de contenu, et unicité de l'email et du nom d'utilisateur sur la table users — défense en profondeur vis-à-vis des contrôles applicatifs.

7.3 Justification SQL / NoSQL et références inter-contextes

PostgreSQL héberge le cœur structuré de l'application : des données fortement liées entre elles (hiérarchie de contenu), soumises à des règles d'intégrité strictes, pour lesquelles les propriétés ACID et l'intégrité référentielle du relationnel sont indispensables. MongoDB héberge les commentaires de leçons : des documents autonomes, au schéma souple, sans jointures, parfaitement adaptés au modèle documentaire.

Les identifiants croisés entre contextes — Result.userId côté PostgreSQL, Comment.userId et Comment.lessonId côté MongoDB — sont des références logiques, volontairement sans clé étrangère physique. Aucune FK n'est d'ailleurs possible entre deux bases hétérogènes. Ce couplage faible (loose coupling) préserve l'indépendance des microservices : chaque service possède son périmètre de données (pattern database-per-service), et l'intégrité inter-contextes est garantie par la logique applicative et la chaîne d'identité décrite en section 10.

7.4 Requêtes SQL significatives

Trois requêtes représentatives, exécutées sur la base PostgreSQL du projet :

1. Jointure — les leçons de chaque chapitre, dans l'ordre pédagogique :

```
SELECT c.title AS chapitre, l.title AS lecon
FROM chapter c
JOIN lesson l ON l.chapter_id = c.id
ORDER BY c.order_index, l.order_index;
```

2. Agrégat — score moyen et nombre de tentatives par quiz :

```
SELECT q.title AS quiz,
       COUNT(r.id) AS tentatives,
       ROUND(AVG(r.score), 1) AS score_moyen
FROM quiz q
LEFT JOIN result r ON r.quiz_id = q.id
GROUP BY q.title
ORDER BY score_moyen DESC NULLS LAST;
```

3. Comptage par utilisateur — quiz joués et réussis :

```
SELECT u.username,
       COUNT(r.id) AS quiz_joues,
       COUNT(*) FILTER (WHERE r.completed) AS quiz_reussis
FROM users u
LEFT JOIN result r ON r.user_id = u.id
GROUP BY u.username
ORDER BY quiz_joues DESC;
```

Exécution des trois requêtes sur la base du projet (console SQL IntelliJ) :

```
SELECT c.title AS chapitre, l.title AS lecon
FROM chapter c
      JOIN lesson l 1<->1..n: ON l.chapter_id = c.id
ORDER BY c.order_index, l.order_index;
```

	chapitre ▼	÷	lecon ▼	÷
1	Introduction à Java		Les variables en Java	
2	Variables et types de données		Déclarer une variable	
3	Les conditions		Le if et le else	
4	Les boucles		La boucle for	

Requête 1 — les leçons de chaque chapitre

```
SELECT q.title AS quiz,
       COUNT(r.id) AS tentatives,
       ROUND(AVG(r.score), 1) AS score_moyen
FROM quiz q
      LEFT JOIN result r 1<->0..n: ON r.quiz_id = q.id
GROUP BY q.title
ORDER BY score_moyen DESC NULLS LAST;
```

	quiz ▼	÷	tentatives ▼	÷	score_moyen ▼	÷
1	Quiz - Les variables		3		77.7	
2	Quiz - Variables et types		1		33	
3	Quiz - La boucle for		0		<null>	

Requête 2 — score moyen et tentatives par quiz (le LEFT JOIN conserve le quiz jamais joué)

```
SELECT u.username,
       COUNT(r.id) AS quiz_joues,
       COUNT(*) FILTER (WHERE r.completed) AS quiz_reussis
FROM users u
      LEFT JOIN result r 1<->0..n: ON r.user_id = u.id
GROUP BY u.username
ORDER BY quiz_joues DESC;
```

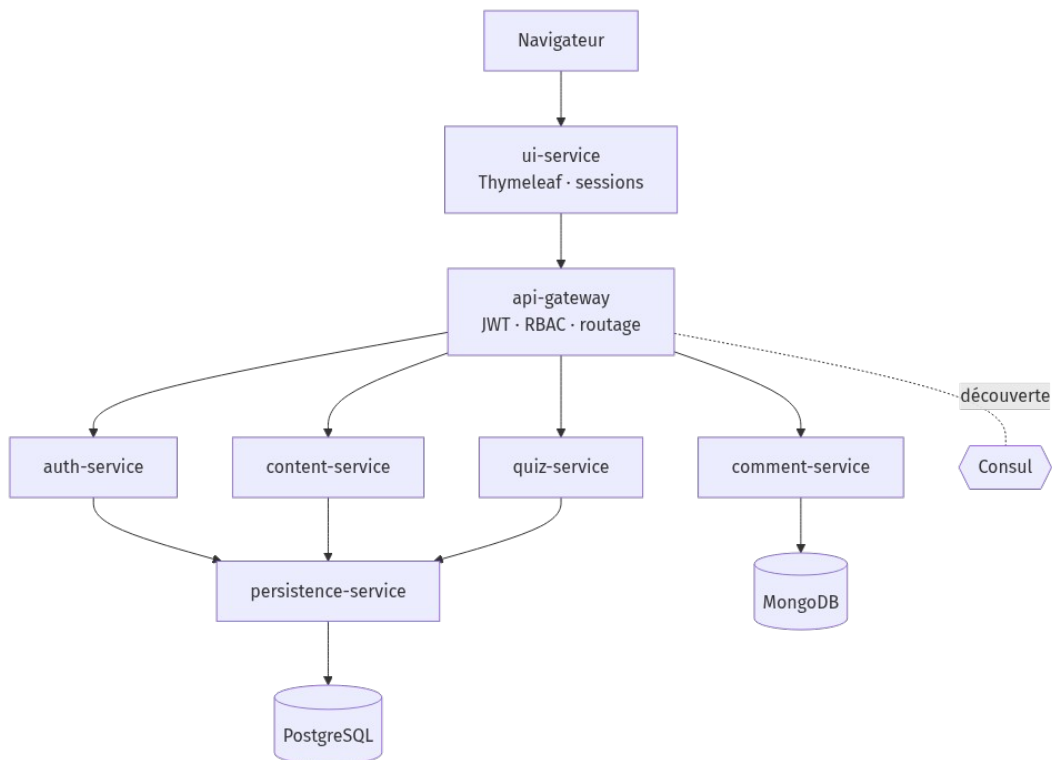
	username ▼	÷	quiz_joues ▼	÷	quiz_reussis ▼	÷
1	demo_user		4		2	
2	popjav_admin		0		0	
3	thomas_j		0		0	
4	marie_dev		0		0	
5	lea_code		0		0	

Requête 3 — quiz joués et réussis par utilisateur

08 — Architecture

8.1 Vue d'ensemble

PopJav est découpé en sept microservices Spring Boot indépendants, qui s'enregistrent auprès de Consul et communiquent en REST via OpenFeign. Le navigateur ne dialogue qu'avec le service d'interface ; toutes les requêtes vers les services métier transitent par la passerelle API, seul point d'entrée qui applique la sécurité.



L'architecture PopJav : le navigateur ne parle qu'au BFF, le gateway est le seul point d'entrée vers les services métier

8.2 Rôle de chaque service

Service	Rôle
ui-service	Backend-for-Frontend (BFF) : pages Thymeleaf, session utilisateur côté serveur, aucun accès direct aux données. Un intercepteur Feign rattache automatiquement le JWT de session à chaque appel sortant.
api-gateway	Point d'entrée unique (Spring Cloud Gateway) : routage, validation des JWT, contrôle d'accès par rôle (RBAC), routes publiques, et propagation d'une identité de confiance (en-têtes X-User-Id / X-User-Role écrasés).
auth-service	Inscription et connexion : politique de mot de passe, hachage BCrypt, message générique anti-énumération, émission des JWT signés (claims userId et role).
content-service	Façade métier du contenu (chapitres, leçons) : expose l'API de consultation dont le résumé public du catalogue ; stockage délégué à persistence-service via

	Feign.
quiz-service	Façade métier des quiz : masque les bonnes réponses au client, calcule les scores côté serveur (vies, pourcentage, seuil), applique le contrôle d'appartenance sur les résultats.
persistence-service	Seul service connecté à PostgreSQL : entités JPA, repositories, API CRUD interne consommée par les autres services.
comment-service	Commentaires de leçons sur MongoDB : documents souples, identité de l'auteur imposée par l'en-tête X-User-Id du gateway.

8.3 Patterns et communication inter-services

L'architecture combine plusieurs patterns classiques : architecture en couches à l'intérieur de chaque service (Controller / Service / Repository), client-serveur entre le BFF et les services métier, service discovery avec Consul (les adresses ne sont jamais codées en dur ; les healthchecks retirent automatiquement un service défaillant), MVC côté ui-service, et clients REST déclaratifs avec OpenFeign.

Pour illustrer la communication, voici le flux complet d'une soumission de quiz :

- 1. Le joueur valide le formulaire ; ui-service récupère son userId depuis la session serveur — jamais depuis le formulaire.
- 2. L'intercepteur Feign ajoute le JWT de session en en-tête Authorization ; la requête part vers le gateway.
- 3. Le gateway valide la signature et l'expiration du token, puis écrase les en-têtes X-User-Id / X-User-Role avec les valeurs extraites du token signé : un client ne peut pas les forger.
- 4. quiz-service remplace le userId du corps de requête par celui de l'en-tête, récupère sa propre copie du quiz (avec les bonnes réponses) auprès de persistence-service, calcule le score et décompte les vies.
- 5. Le résultat est enregistré via persistence-service, puis renvoyé à ui-service qui affiche la correction.

09 — Développement : extraits de code commentés

Les extraits ci-dessous sont tirés du code réel du projet, avec leurs commentaires d'origine (le code est commenté en anglais).

9.1 Front-end — fragments Thymeleaf et formulaires

L'en-tête HTML est un fragment paramétré par le titre de la page, réutilisé par tous les templates : une seule source de vérité pour les feuilles de style (versionnées pour le cache-busting) et les polices :

```
<head th:fragment="header(title)">
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title th:text="${title}">PopJav</title>
  <link rel="stylesheet" th:href="@{/css/style.css?v=6}">
  <link rel="stylesheet" th:href="@{/css/forms.css?v=6}">
  ...
</head>
```

Les formulaires sont liés aux DTO côté serveur avec `th:object` / `th:field` ; Thymeleaf gère le binding des champs et la régénération des valeurs :

```
<form th:action="@{/chapters/create}" th:object="${chapterDTO}" method="post">
  <label for="title" class="form-label">Titre</label>
  <input type="text" th:field="*{title}" class="form-input">
  <label for="description" class="form-label">Description</label>
  <textarea th:field="*{description}" class="form-input form-textarea"></textarea>
  <button type="submit" class="form-btn">Créer</button>
</form>
```

9.2 Back-end — le moteur de quiz

Le calcul de score est la logique métier la plus riche du projet. Le service récupère sa propre copie du quiz — on ne fait jamais confiance à un score calculé par le client — puis évalue les réponses en décomptant les vies :

```
// Re-fetches its own copy of the quiz with the "correct" flags intact and
// calculates the score server-side: never trust a user to calculate the score.
public QuizResultDTO submitQuiz(QuizSubmissionDTO submission) {
    QuizDTO quiz = quizFeignClient.getQuizById(submission.getQuizId());
    int score = 0;
    int lives = quiz.getLives();
    for (QuestionDTO question : quiz.getQuestions()) {
        ...
        // Every wrong answer costs a life
        if (isCorrect) { score++; } else { lives--; }
        ...
        // When lives reach 0 the game stops: the remaining questions are not
        // evaluated. That is the rule of the game, not a bug.
        if (lives <= 0) { break; }
    }
}
```

```

    // A quiz without any question must not break the percentage calculation
    int totalQuestions = quiz.getQuestions().size();
    int percentage = totalQuestions == 0 ? 0 : (score * 100) / totalQuestions;
    boolean passed = percentage >= quiz.getPassingScore();
    ...
}

```

Son pendant défensif : avant tout envoi au client, les bonnes réponses sont masquées — un joueur qui inspecterait la réponse réseau ne verrait que des « correct: false ».

9.3 Back-end — la chaîne d'identité

Côté BFF, un intercepteur rattache automatiquement le JWT de session à chaque appel Feign sortant :

```

/**
 * Re-attaches the session JWT as a Bearer token on every outgoing Feign call,
 * otherwise the gateway would answer 401. The browser never sees the JWT:
 * it only knows the session cookie.
 */
@Component
public class FeignInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        ...
        String token = (String) session.getAttribute("token");
        if (token != null) {
            template.header("Authorization", "Bearer " + token);
        }
    }
}

```

Côté gateway, l'identité est extraite du token signé puis propagée aux services aval en écrasant les en-têtes entrants :

```

// The identity comes from the signed token, not from the client:
// incoming values are overwritten so a client can never forge them
String userId = jwtService.extractUserId(token);
ServerHttpRequest request = exchange.getRequest().mutate()
    .headers(headers -> {
        headers.remove("X-User-Id");
        headers.remove("X-User-Role");
        if (userId != null) { headers.set("X-User-Id", userId); }
        headers.set("X-User-Role", jwtService.extractRole(token));
    })
    .build();

```

9.4 Le pattern DTO comme barrière de sécurité

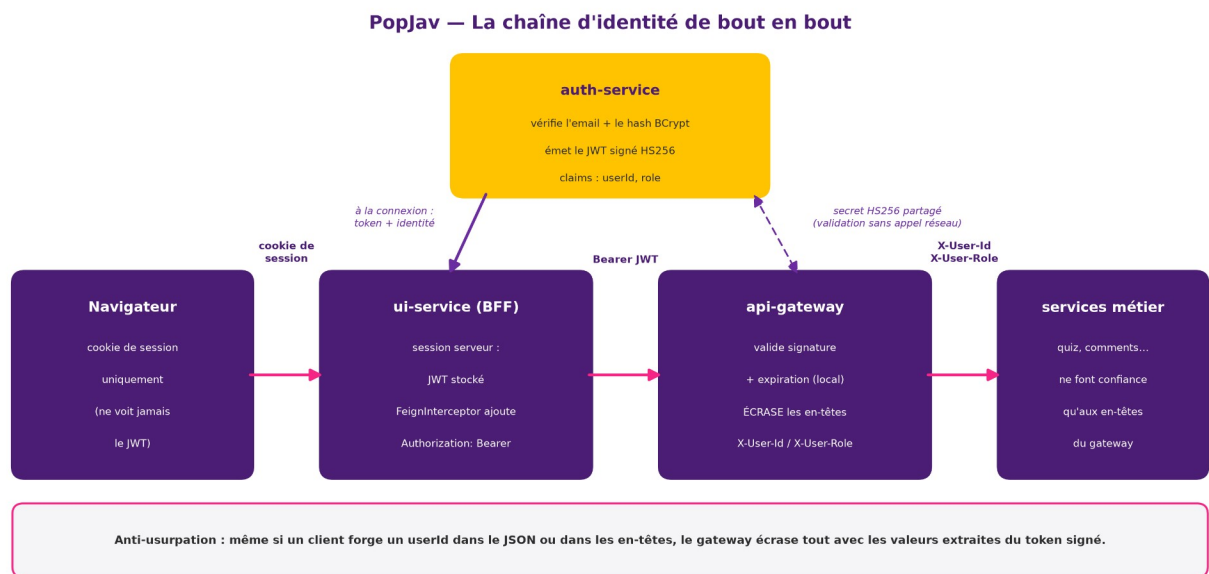
Les entités JPA ne sont jamais exposées telles quelles. Côté utilisateurs, toutes les réponses de l'API passent par un DTO qui ne possède pas de champ mot de passe : la fuite du hash est impossible par construction, et non par simple convention :

```
// All the responses go through UserResponseDTO: the password hash can never
// leak since the DTO has no password field. The only deliberate exception
// is /credentials.
private UserResponseDTO toResponse(User user) {
    return new UserResponseDTO(
        user.getId(), user.getUsername(), user.getEmail(),
        user.getRole(), user.getCreatedAt());
}
```


10 — Sécurité de l'application

10.1 Authentification et chaîne d'identité

La sécurité de PopJav repose sur une chaîne d'identité de bout en bout : à la connexion, auth-service vérifie les identifiants (hash BCrypt) et émet un JWT signé HS256 contenant les claims `userId` et `role`. Ce token est stocké dans la session côté serveur du BFF — le navigateur ne le voit jamais, il ne possède qu'un cookie de session. À chaque appel vers les services métier, l'intercepteur Feign transmet le token ; le gateway le valide (signature + expiration, localement grâce au secret partagé, sans appel réseau) puis propage l'identité via les en-têtes `X-User-Id` / `X-User-Role`, en écrasant systématiquement les valeurs entrantes. Les services aval font confiance à ces en-têtes et à eux seuls : même un JSON forgé avec le `userId` d'autrui est ignoré.



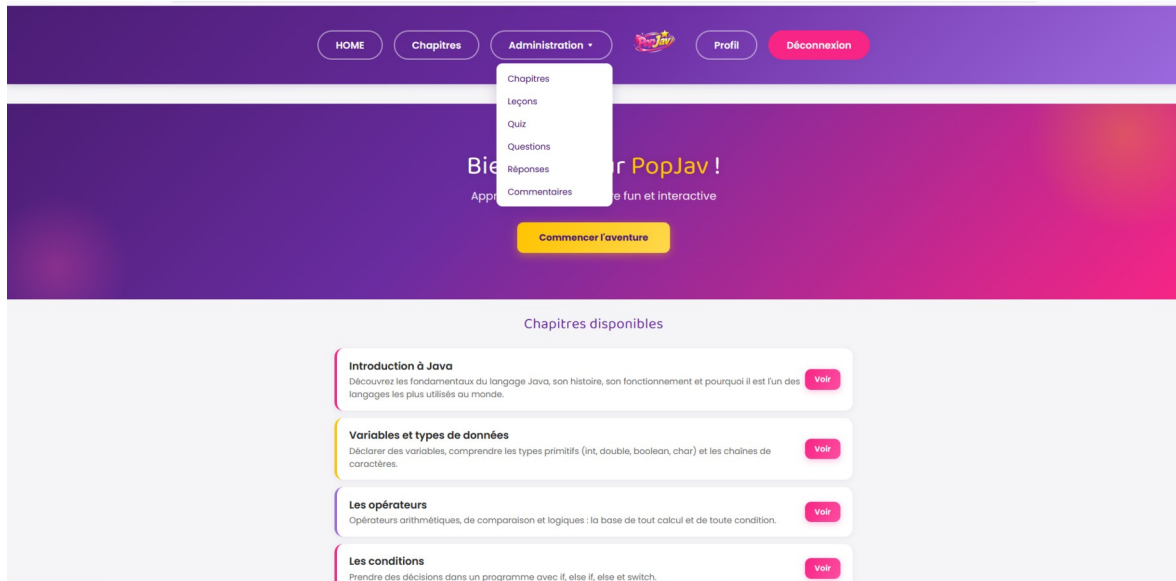
La chaîne d'identité de bout en bout

10.2 Mesures en place

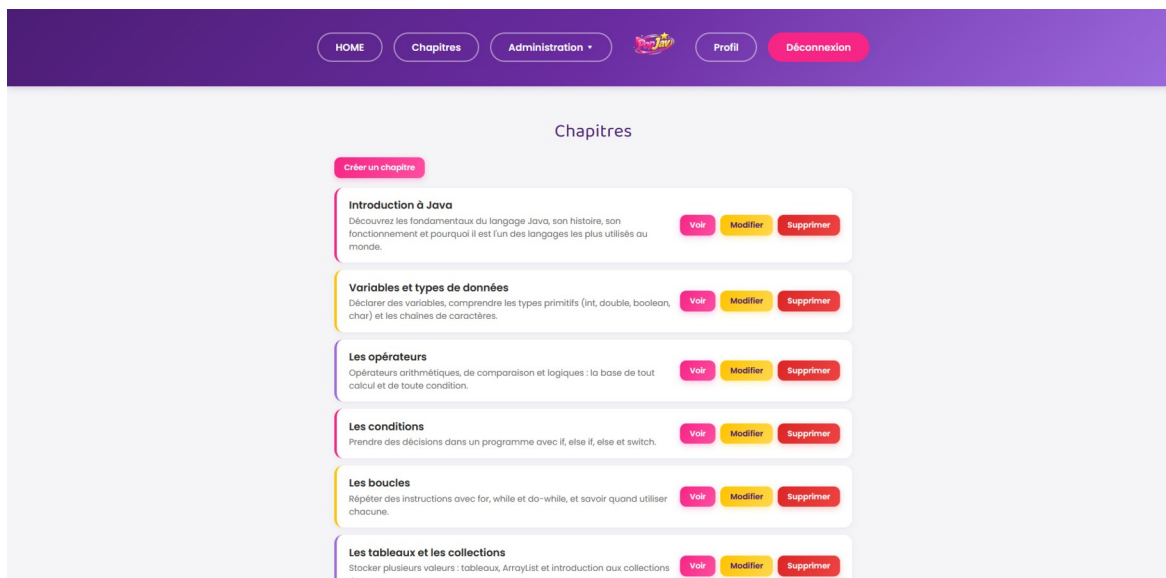
- Hachage BCrypt côté auth-service : le mot de passe en clair ne quitte jamais ce service ; la base ne voit que le hash.
- Politique de mot de passe validée côté serveur (longueur, majuscule, minuscule, chiffre, caractère spécial) : la validation du front peut être contournée en appelant l'API directement, pas celle du serveur.
- Message d'erreur générique à la connexion : email inconnu et mot de passe erroné renvoient la même réponse, la plateforme ne révèle pas si un compte existe (anti-énumération).
- RBAC au gateway : les écritures sur le contenu et l'administration des utilisateurs sont réservées au rôle ADMIN ; la route interne `/credentials` (qui renvoie le hash) est inaccessible à un utilisateur normal.

- Anti-IDOR : un utilisateur ne peut lire que ses propres résultats ; un admin peut tout lire ; toute autre tentative reçoit un 403.
- Anti-triche : les bonnes réponses ne sont jamais envoyées au client ; le scoring est exclusivement côté serveur.
- DTO sans champ password : aucune réponse d'API ne peut exposer le hash, par construction.
- Injections SQL : toutes les requêtes passent par Spring Data JPA / Hibernate, donc par des requêtes préparées ; unicité email/username garantie par la base (défense en profondeur).
- API stateless : pas de cookie de session côté services, donc pas de surface CSRF ; chaque requête porte son JWT.

Côté interface, le RBAC est visible : les actions d'écriture ne sont proposées qu'aux comptes ADMIN (le gateway les refuse de toute façon à un USER, même en forgeant la requête) :



Le menu Administration, visible uniquement pour un compte ADMIN



La gestion des chapitres : les actions Modifier/Supprimer sont réservées à l'ADMIN

10.3 Correspondance OWASP Top 10

Risque OWASP (2021)	Réponse dans PopJav
A01 — Broken Access Control	RBAC au gateway (écritures admin, routes internes) + contrôle d'appartenance sur les résultats (anti-IDOR) + identité imposée par le token signé.
A02 — Cryptographic Failures	Hachage BCrypt (salé, coût adaptatif) ; JWT signés HS256 ; aucun mot de passe en clair stocké ou transmis entre services.
A03 — Injection	Requêtes préparées générées par Spring Data JPA/Hibernate ; validation des entrées côté serveur.
A05 — Security Misconfiguration	Secrets externalisés dans le fichier .env (jamais dans le code) ; les services ne communiquent avec les bases qu'à travers le réseau Docker interne — leurs ports ne sont publiés sur la machine hôte qu'en développement, pour l'outillage local, et ne le seraient pas en production.
A07 — Identification & Authentication Failures	Politique de mot de passe serveur ; message générique anti-énumération ; expiration des tokens ; validation systématique au gateway.

11 — Conteneurisation et déploiement

Les sept services sont conteneurisés avec Jib, un plugin Maven qui construit des images Docker optimisées sans Dockerfile : les dépendances, les ressources et les classes sont placées dans des couches distinctes, ce qui rend les reconstructions incrémentales très rapides et standardise la production des images. Les images sont publiées sur Docker Hub.

Un fichier docker-compose unique orchestre l'ensemble de la plateforme — dix conteneurs : les sept services applicatifs, PostgreSQL 16, MongoDB 7 et Consul. Les dépendances de démarrage sont explicites : des healthchecks vérifient que PostgreSQL, MongoDB et Consul sont réellement prêts (pg_isready, ping mongosh, API Consul) avant que les services applicatifs ne démarrent (depends_on + condition: service_healthy), ce qui évite les échecs de connexion au lancement.

La configuration est externalisée dans un fichier .env (ports, identifiants de bases, secret JWT) : aucune valeur sensible dans les images ni dans le code. La procédure de lancement complète tient en deux étapes :

```
# 1. Renseigner la configuration
cp .env.example .env # puis compléter les valeurs

# 2. Lancer la plateforme complète
docker compose up -d
```

```
PS C:\Users\Hp\IdeaProjects\PopJav> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
api-gateway	enzogavini/api-gateway:latest	"java -cp @/app/jib-..."	api-gateway	2 hours ago	Up 2 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
auth-service	enzogavini/auth-service:latest	"java -cp @/app/jib-..."	auth-service	2 hours ago	Up 2 hours	
comment-service	enzogavini/comment-service:latest	"java -cp @/app/jib-..."	comment-service	2 hours ago	Up 2 hours	
consul	hashicorp/consul:1.21.3	"docker-entrypoint.s..."	consul	2 hours ago	Up 2 hours (healthy)	0.0.0.0:8500->8500/tcp, [::]:8500->8500/tcp
content-service	enzogavini/content-service:latest	"java -cp @/app/jib-..."	content-service	2 hours ago	Up 2 hours	
mongo-popjav	mongo:7	"docker-entrypoint.s..."	mongo	2 hours ago	Up 2 hours (healthy)	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
persistence-service	enzogavini/persistence-service:latest	"java -cp @/app/jib-..."	persistence-service	2 hours ago	Up 2 hours	
postgres-popjav	postgres:16	"docker-entrypoint.s..."	postgres	2 hours ago	Up 2 hours (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
quiz-service	enzogavini/quiz-service:latest	"java -cp @/app/jib-..."	quiz-service	2 hours ago	Up 2 hours	
ui-service	enzogavini/ui-service:latest	"java -cp @/app/jib-..."	ui-service	2 hours ago	Up 2 hours	0.0.0.0:8085->8085/tcp, [::]:8085->8085/tcp

```
PS C:\Users\Hp\IdeaProjects\PopJav>
```

Les 10 conteneurs de la plateforme (docker compose ps)

Services

8 total

Q Search Search Across	Health Status	Service Type	Unhealthy to Healthy
✓ consul 1 instance			
✓ api-gateway 1 instance			
✓ auth-service 1 instance			
✓ comment-service 1 instance			
✓ content-service 1 instance			
✓ persistence-service 1 instance			
✓ quiz-service 1 instance			
✓ ui-service 1 instance			

Le dashboard Consul : tous les services enregistrés et sains

12 — Tests

12.1 Stratégie et outillage

La stratégie de test cible en priorité la logique métier — les services, là où vivent les règles du jeu et de sécurité — puis sécurise les couches d'entrée : contrôleurs (règles d'accès, propagation d'identité) et contrats des DTO. Les tests unitaires utilisent JUnit 5 et Mockito : les dépendances externes (clients Feign, PasswordEncoder, JwtService) sont simulées, ce qui isole la logique testée de l'état des autres services. L'exécution est intégrée au build Maven.

12.2 Couverture

La couverture est mesurée par JaCoCo : 89 % des instructions sont couvertes sur quiz-service et 88 % sur auth-service. Les couches service et controller sont couvertes à près de 100 % ; les DTO — du code généré par Lombok — le sont à 84-86 % grâce à un test de contrat générique. Les branches restantes non couvertes proviennent essentiellement des méthodes equals() générées par Lombok sur les DTO ; sur la logique métier, tous les chemins critiques — vies à zéro, seuil de réussite, division par zéro — sont testés.

quiz-service

quiz-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.enzo.quizservice.dto		86 %		46 %	114 231	0 41	0 103	0 7
com.enzo.quizservice		37 %		n/a	1 2	2 3	1 2	0 1
com.enzo.quizservice.service		99 %		83 %	4 41	0 79	0 29	0 4
com.enzo.quizservice.controller		100 %		100 %	0 28	0 33	0 26	0 4
Total	207 of 1 933	89 %	141 of 284	50 %	119 302	2 156	1 160	0 16

Rapport JaCoCo de quiz-service : 89 % des instructions couvertes

auth-service

auth-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.enzo.authservice.dto		84 %		44 %	102 192	0 30	0 78	0 6
com.enzo.authservice		37 %		n/a	1 2	2 3	1 2	0 1
com.enzo.authservice.service		100 %		77 %	4 15	0 63	0 6	0 2
com.enzo.authservice.config		100 %		n/a	0 11	0 14	0 11	0 2
com.enzo.authservice.exception		100 %		n/a	0 4	0 8	0 4	0 4
com.enzo.authservice.controller		100 %		n/a	0 3	0 3	0 3	0 1
com.enzo.authservice.entity		100 %		n/a	0 1	0 3	0 1	0 1
Total	189 of 1 578	88 %	130 of 246	47 %	107 228	2 124	1 105	0 17

Rapport JaCoCo d'auth-service : 88 % des instructions couvertes

12.3 Tests représentatifs

Côté authentification (AuthServiceTest, AuthServiceLoginTest) :

- Inscription réussie : le mot de passe transmis à la persistance est bien le hash, jamais le clair, et un token est renvoyé.
- Email ou nom d'utilisateur déjà pris : l'exception typée attendue est levée et rien n'est enregistré.

- Politique de mot de passe et format d'email invalides : rejetés côté serveur.
- Connexion réussie : token émis avec l'identité (userId, rôle).
- Anti-énumération vérifiée : email inconnu et mot de passe erroné lèvent exactement la même exception générique.

Côté moteur de quiz (QuizServiceTest, QuizGameRulesTest) :

- Un sans-faute donne 100 %, le quiz est réussi et le résultat enregistré.
- Décompte des vies et arrêt immédiat de la partie à zéro vie : les questions restantes ne sont pas évaluées.
- Score sous le seuil de réussite → quiz échoué, score conservé.
- Quiz sans questions → pas de division par zéro.
- Les bonnes réponses sont masquées avant tout envoi au client (correct = false partout).

Côté couches d'entrée et transverses :

- ResultControllerTest : les règles d'appartenance — le propriétaire lit ses résultats, l'admin lit tout, tout autre appelant reçoit un 403.
- QuizControllerTest : le userId du corps de requête est bien écrasé par l'en-tête X-User-Id du gateway.
- GlobalExceptionHandlerTest : chaque exception métier est convertie vers son code HTTP (409, 401, 400, 500).
- JwtServiceTest : le token émis est un JWT signé en trois parties.
- DtoTest : les contrats equals/hashCode et les accesseurs de tous les DTO (code généré par Lombok) sont exercés.

13 — Recette et validation

13.1 Grille de recette fonctionnelle

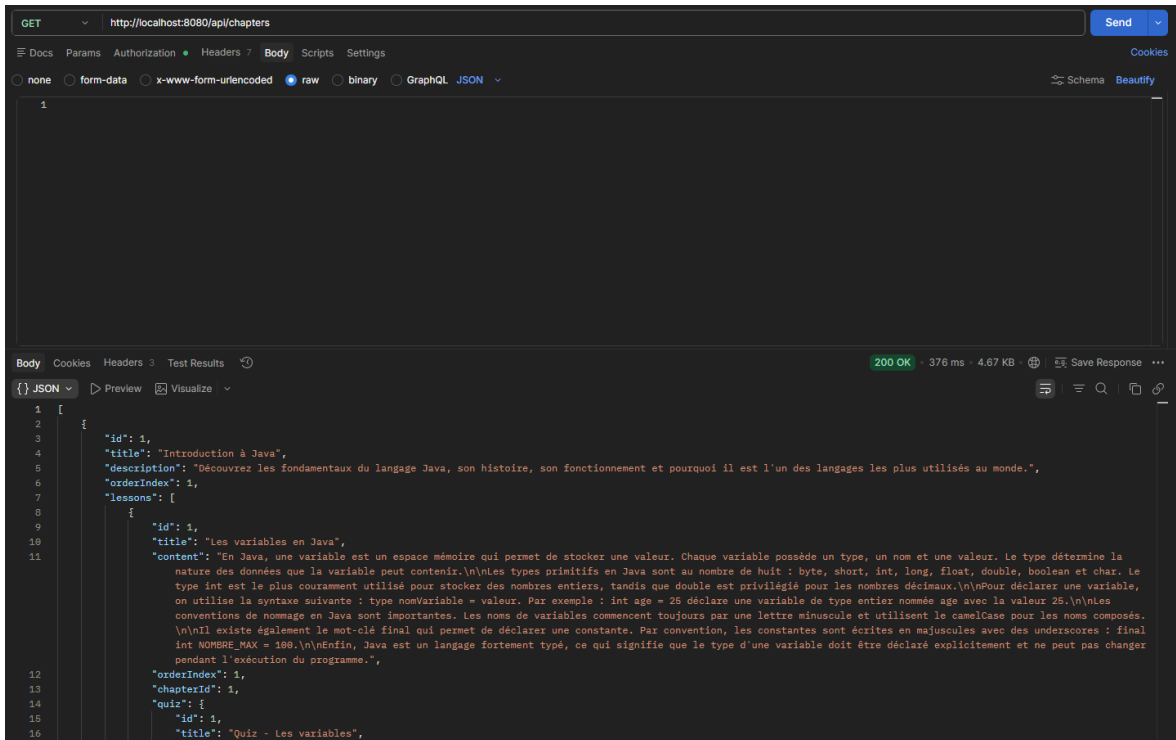
Chaque scénario simule un comportement utilisateur de bout en bout sur l'interface :

ID	Module	Fonctionnalité	Action réalisée	Résultat attendu	Statut
TC-01	auth-service	Inscription	Saisie username, email, mot de passe valide	Compte créé, mot de passe haché en base, connexion automatique	OK
TC-02	auth-service	Robustesse inscription	Email déjà existant / mot de passe trop faible	Blocage avec message d'erreur clair, pas de crash	OK
TC-03	auth-service	Anti-énumération	Connexion avec email inconnu puis avec mauvais mot de passe	Le même message générique dans les deux cas	OK
TC-04	quiz-service	Quiz complet	Jouer un quiz jusqu'au bout avec des erreurs	Score en %, vies décomptées, correction affichée, résultat enregistré	OK
TC-05	quiz-service	Fin de partie	Épuiser toutes les vies	La partie s'arrête immédiatement, résultat non réussi	OK
TC-06	api-gateway	RBAC admin	Accès à une action d'administration avec un compte USER	Refus (403), l'action n'est pas visible dans l'interface	OK
TC-07	quiz-service	Anti-IDOR	Tentative de lecture des résultats d'un autre utilisateur	Refus (403 Access denied)	OK
TC-08	comment-service	Commentaires	Poster un commentaire sur une leçon	Commentaire enregistré avec l'identité de la session, affiché sous la leçon	OK

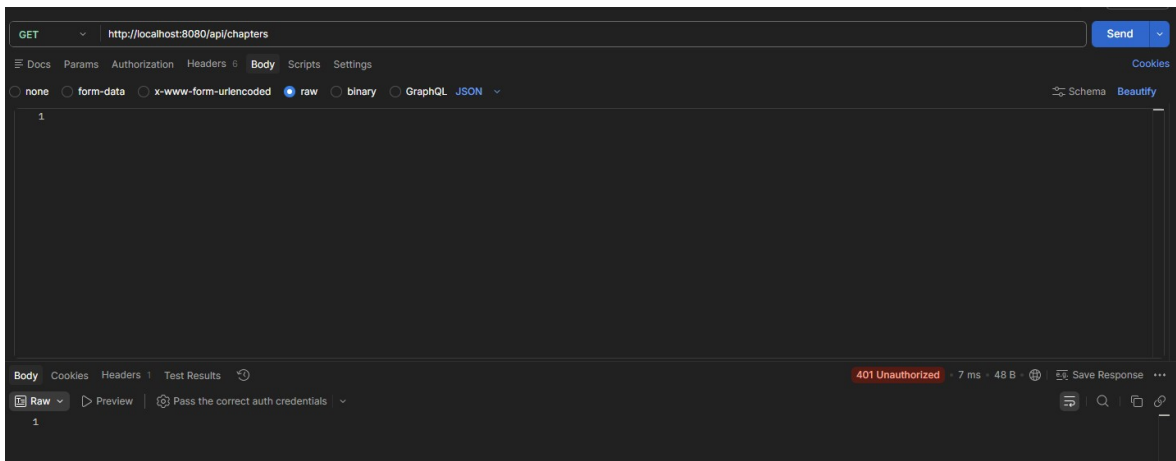
13.2 Tests d'API (Postman, via le gateway)

Les endpoints ont été testés via la passerelle : les codes d'erreur attendus (401, 403) sont aussi importants que les 200 — ils prouvent que la sécurité fonctionne.

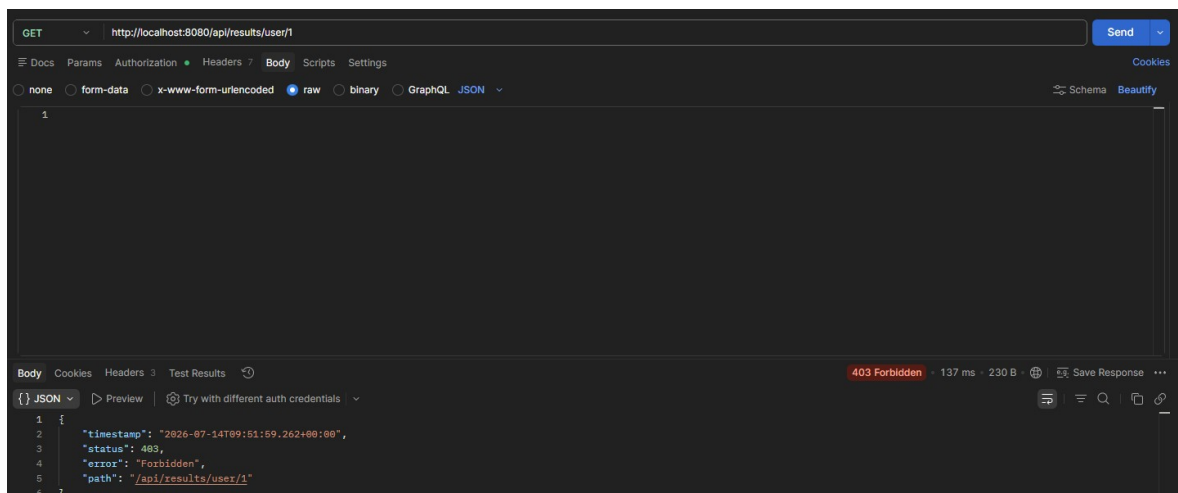
ID	Objectif	Méthode	Endpoint	Contexte	Code attendu
PT-01	Inscription	POST	/auth/register	Public	200



Lecture du contenu avec un Bearer token valide : 200 OK



La même requête sans token : 401 Unauthorized — le gateway bloque



Lecture des résultats d'un autre utilisateur avec un token valide : 403 Forbidden (anti-IDOR)

14 — Conformité RGPD et accessibilité (RGAA)

14.1 RGPD

Le tableau ci-dessous met en correspondance les principes du RGPD avec leur implémentation technique dans PopJav :

Principe RGPD	Exigence	Implémentation dans PopJav
Minimisation des données (art. 5)	Ne collecter que le strict nécessaire	La table users se limite à username, email, mot de passe (haché) et rôle ; aucun tracking, aucune donnée superflue.
Droit d'accès (art. 15)	L'utilisateur consulte ses données	La page profil affiche les informations du compte et l'historique des résultats.
Sécurité et privacy by design (art. 25 et 32)	Protection dès la conception	BCrypt, DTO sans champ password, secret JWT externalisé ; les services accèdent aux bases via le réseau Docker interne (ports publiés sur l'hôte uniquement en développement).

14.2 Accessibilité (RGAA)

Le RGAA s'articule autour de quatre principes : perceptible, utilisable, compréhensible et robuste. Les mesures suivantes sont en place dans PopJav :

- Étiquettes explicites (label) sur tous les champs de formulaire, et messages d'erreur annoncés aux lecteurs d'écran (role="alert", aria-invalid).
- Contrastes texte/fond suffisants et indicateur de focus clavier très visible (liseré or de 3 px sur tous les éléments interactifs).
- Navigation complète au clavier (y compris le menu mobile, avec aria-expanded, et la fermeture par Échap) et lien d'évitement vers le contenu principal (#main-content) sur toutes les pages.
- Attributs alternatifs sur les images, hiérarchie de titres cohérente.
- Interface responsive : le contenu reste utilisable du mobile aux grands écrans.

Axes d'amélioration envisagés : audit complet avec Lighthouse/Axe et enrichissement des attributs ARIA sur les composants dynamiques.

15 — Veille technologique et sécurité

Pendant le projet, j'ai pratiqué une veille régulière sur deux axes : la sécurité applicative (OWASP Top 10, vulnérabilités des dépendances) et l'écosystème Spring (notes de version de Spring Boot et Spring Cloud, compatibilités de versions).

Mes sources principales sont Baeldung — la référence de l'écosystème Java/Spring, dont les articles m'ont accompagné notamment sur Spring Security, OpenFeign et les tests — ainsi que les documentations officielles de Spring, Docker et Jib, complétées par les publications de l'OWASP pour le volet sécurité. Cette veille se fait essentiellement en anglais.

Elle a eu des effets concrets sur le projet : la mise en correspondance des mesures de sécurité avec l'OWASP Top 10 (section 10.3), le choix d'un JDK LTS, et le suivi des versions des images Docker utilisées (PostgreSQL 16, MongoDB 7, Consul).

16 — Difficultés rencontrées et évolutions futures

16.1 Difficultés et solutions

La récursion infinie de sérialisation JSON : les relations bidirectionnelles JPA (Quiz ↔ Question ↔ Answer, Quiz ↔ Result) provoquaient une boucle infinie au moment de sérialiser les entités en JSON — le quiz sérialise ses questions, qui re-sérialisent le quiz, à l'infini. J'ai résolu le problème avec les annotations `@JsonManagedReference` (côté parent) et `@JsonBackReference` (côté enfant), qui coupent le cycle à la sérialisation tout en conservant les relations en base.

La propagation de l'identité entre microservices : chaque service doit connaître l'utilisateur appelant sans lui faire confiance. Ma première approche — transmettre le `userId` dans le corps des requêtes — était vulnérable à la falsification. La solution finale est la chaîne d'identité : le gateway extrait l'identité du JWT signé et l'impose aux services aval via des en-têtes écrasés. C'est le point qui m'a le plus appris en matière de sécurité distribuée.

L'orchestration du démarrage : au lancement du `docker-compose`, les services applicatifs plantaient lorsque les bases de données n'étaient pas encore prêtes à accepter des connexions. J'ai résolu le problème avec des healthchecks (`pg_isready` pour PostgreSQL, `ping` pour MongoDB, API de statut pour Consul) combinés à `depends_on` avec condition `service_healthy` : chaque service attend que ses dépendances soient réellement opérationnelles, pas seulement démarrées.

16.2 Évolutions envisagées

- Factorisation des templates de formulaires : fusionner chaque paire création/édition en un template unique piloté par la présence de l'identifiant (déjà identifiée et chiffrée, volontairement priorisée après la certification pour préserver la stabilité).
- Intégration continue : pipeline GitHub Actions (build, tests, construction des images Jib).
- Gamification enrichie : badges, séries, classement.
- Éditeur de contenu riche pour les leçons côté administration.
- Affichage du nom de l'auteur sur les commentaires : le `userId` est déjà stocké dans le document MongoDB, il resterait à résoudre le nom via `persistence-service`.
- Formatage des dates d'affichage (les horodatages sont actuellement affichés au format technique ISO).

17 — Conclusion

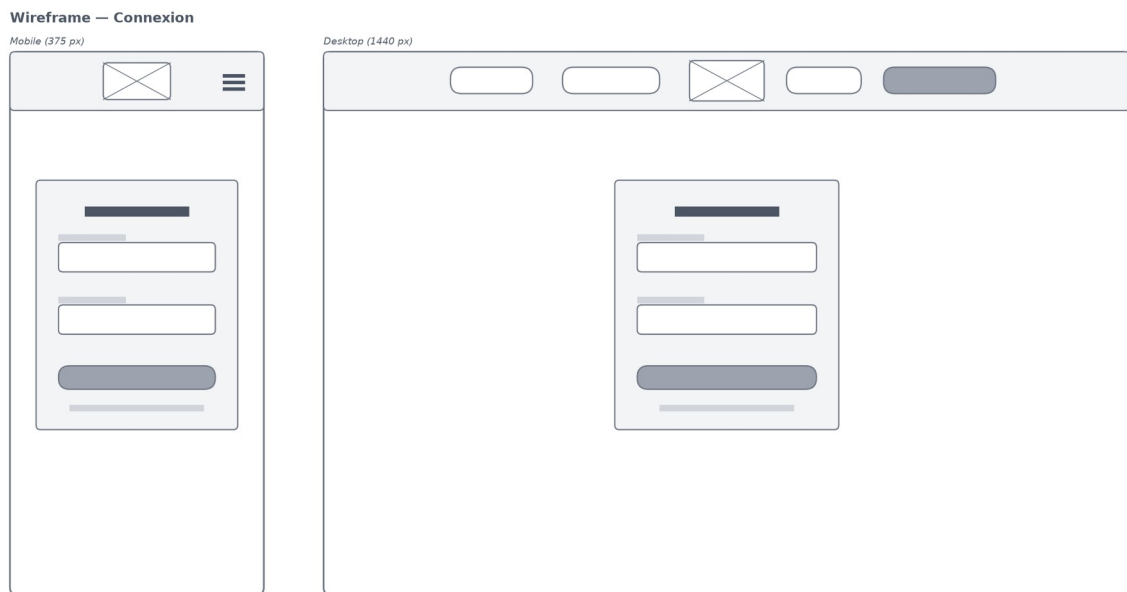
PopJav m'a permis de mettre en œuvre l'ensemble des compétences du titre professionnel sur un projet complet et réaliste : une architecture distribuée de sept microservices Spring Boot, une persistance polyglotte PostgreSQL/MongoDB, une chaîne de sécurité de bout en bout (BCrypt, JWT, RBAC, anti-IDOR), des interfaces responsive et accessibles, des tests unitaires avec une couverture de 88 à 89 %, et un déploiement entièrement conteneurisé.

Au-delà de la technique, ce projet m'a appris à raisonner en développeur professionnel : justifier chaque choix d'architecture, penser la sécurité dès la conception plutôt qu'après coup, prioriser — choisir de stabiliser plutôt que de refactorer à l'approche d'une échéance — et documenter pour les autres : le code est commenté en anglais, et le déploiement est reproductible par n'importe qui en deux commandes.

Ce titre professionnel est la première étape de ma reconversion : je souhaite poursuivre vers le titre de Concepteur Développeur d'Applications, et pourquoi pas jusqu'au master, avec l'objectif de faire du développement logiciel mon métier. Je remercie les membres du jury pour le temps et l'attention consacrés à ce dossier.

18 — Annexes

- Annexe A — Diagramme d'architecture complet (pleine page).
- Annexe B — MCD et MLD (pleine page).
- Annexe C — Lien vers le dépôt GitHub et extrait du README.
- Annexe D — Rapport de couverture JaCoCo.
- Annexe E — docker-compose.yml complet.
- Annexe F — Wireframes complémentaires et captures d'écran de l'application (desktop et mobile).



Wireframe — connexion

Wireframe — Page de leçon (contenu, quiz, commentaires)

Mobile (375 px)



Desktop (1440 px)



Wireframe — page de leçon

Wireframe — Profil et progression

Mobile (375 px)

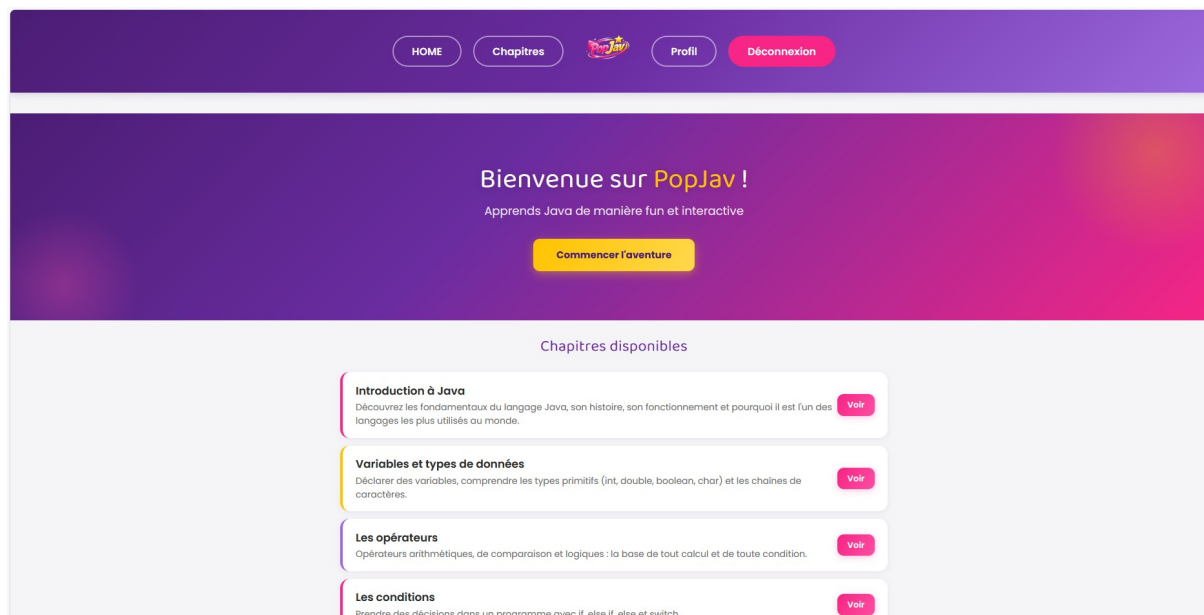


Desktop (1440 px)

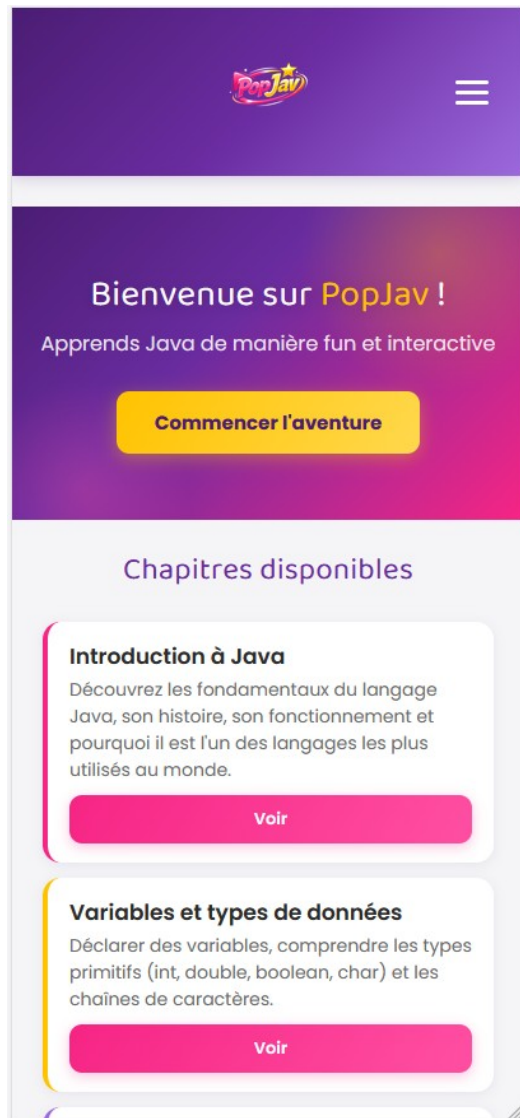


Wireframe — profil

Annexe F — Captures d'écran de l'application :



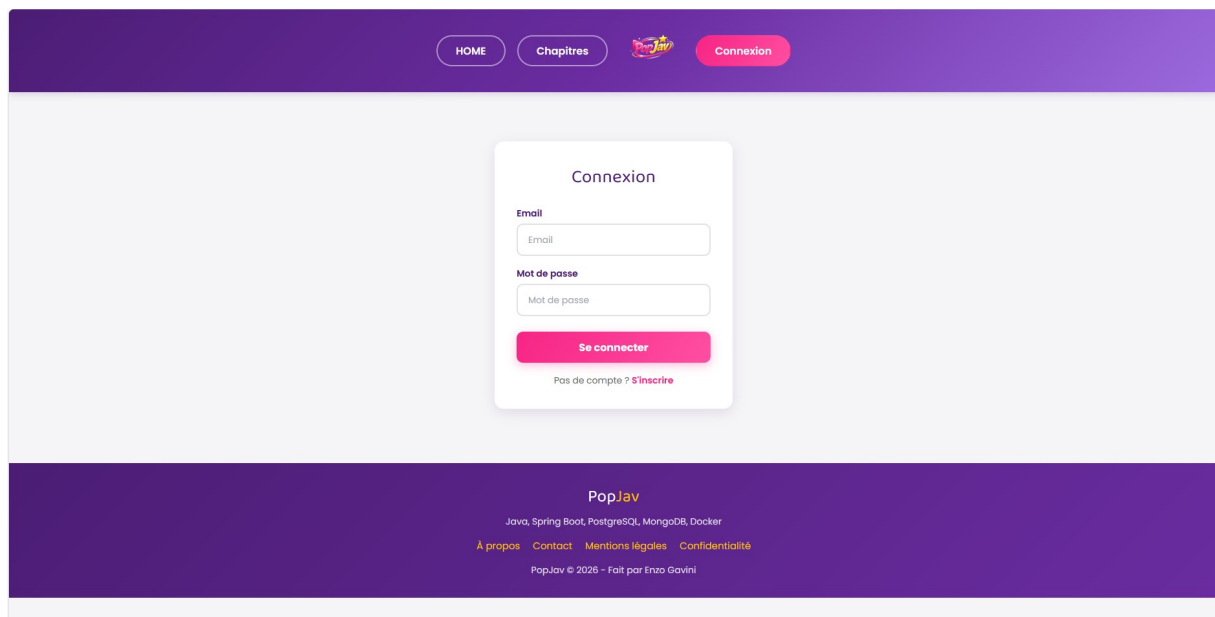
L'accueil (desktop) : le hero et le catalogue des chapitres



L'accueil en mobile



Le menu burger mobile déployé (aria-expanded géré au clavier)



La page de connexion et le pied de page



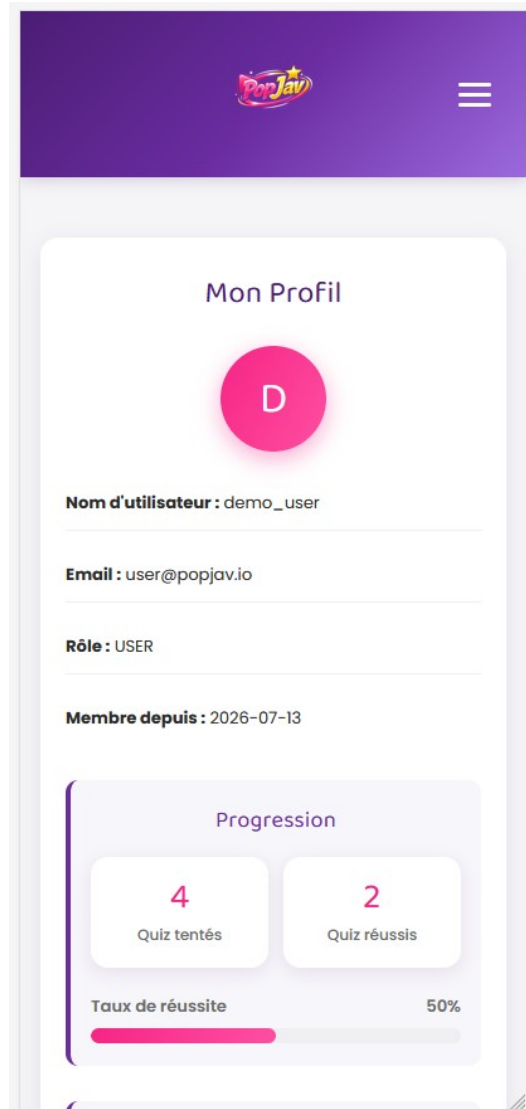
La liste des chapitres en mobile



Les variables en Java

En Java, une variable est un espace mémoire qui permet de stocker une valeur. Chaque variable possède un type, un nom et une valeur. Le type détermine la nature des données que la variable peut contenir. Les types primitifs en Java sont au nombre de huit : byte, short, int, long, float, double, boolean et char. Le type int est le plus couramment utilisé pour stocker des nombres entiers, tandis que double est privilégié pour les nombres décimaux. Pour déclarer une variable, on utilise la syntaxe suivante : `type nomVariable = valeur`. Par exemple : `int age = 25` déclare une variable de type entier nommée age avec la valeur 25. Les conventions de nommage en Java sont importantes. Les noms de variables commencent toujours par une lettre minuscule et utilisent le camelCase pour les noms composés. Il existe également le mot-clé final qui permet de déclarer une constante. Par convention, les constantes sont écrites en majuscules avec des underscores : `final int NOMBRE_MAX = 100`.

Une leçon en mobile



Le profil : statistiques de progression calculées depuis les résultats

Quiz - Les variables Jouer

Commentaires

Votre commentaire

Écrire un commentaire

Commenter

Très bonne introduction, les exemples sont clairs !
2026-07-13T09:53:46.843

Le passage sur le camelCase m'a bien aidé, merci.
2026-07-13T09:53:47.111

Leçon au top, je passe au quiz !
2026-07-13T09:53:47.178

Retour

Les commentaires d'une leçon (stockés dans MongoDB) et le formulaire de publication

HOME Chapitres Administration Profil Déconnexion

Créer un chapitre

Titre

Titre

Description

Description

Ordre

0

Créer

[Retour aux chapitres](#)

Le formulaire de création d'un chapitre (ADMIN)